MANCHESTER 1824
The University of Manchester

On How Transformers Recognise Quantifier Depth in First-order logic

Author: Mei Jia

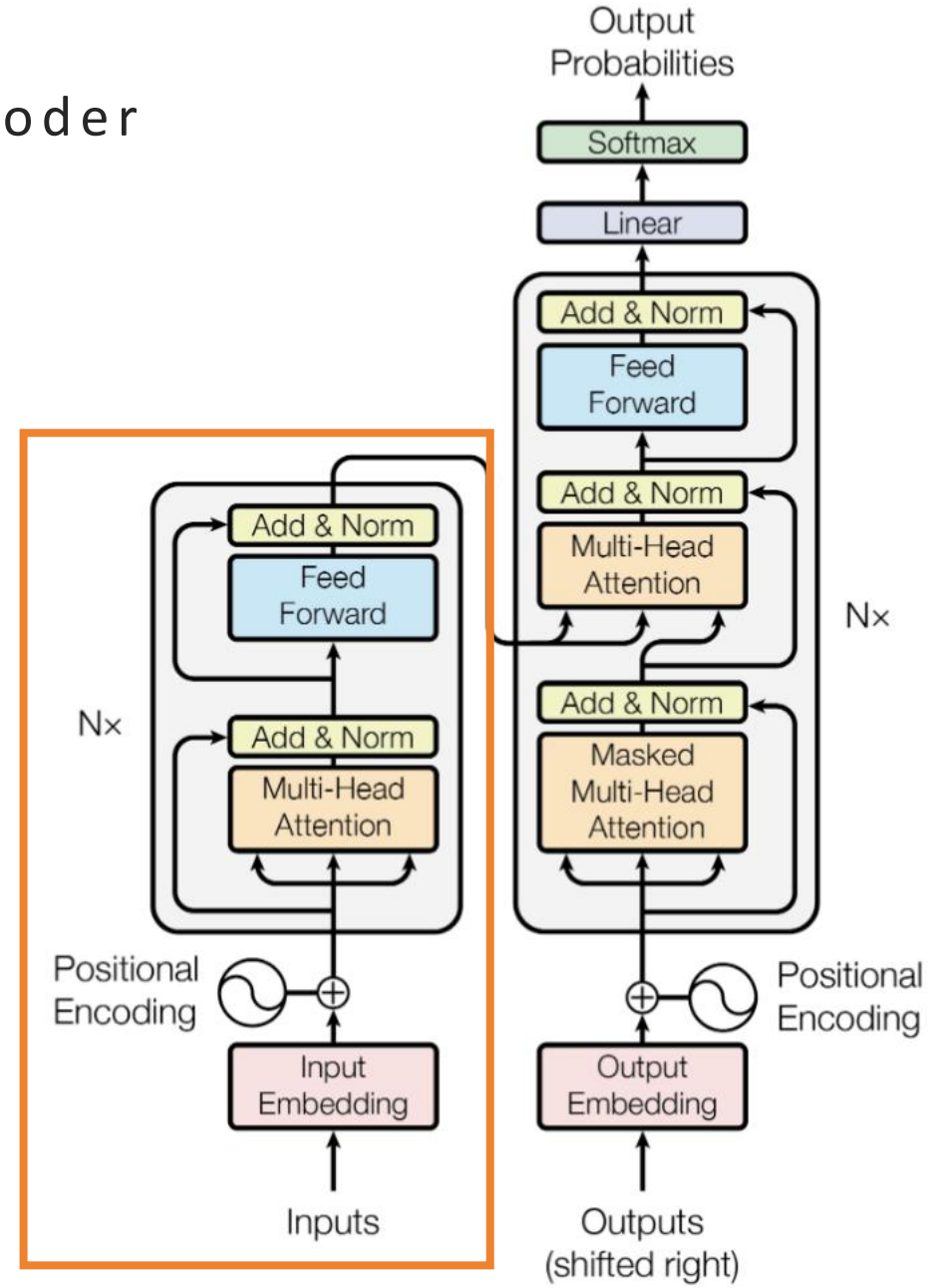
Supervisor: Dmitry Nikolaev

MSc Computational and Corpus Linguistics, LEL SALC, UoM

01/10/2025

Preliminaries

1-1 Transformer Encoder



1-2 First-Order Logic (FOL)

- A logical system for reasoning about properties of objects or relationships between objects in a domain.

$$\Phi \to P(t_{1:m}) \mid (\Phi \wedge \Phi) \mid (\Phi \vee \Phi) \mid (\Phi \to \Phi) \mid \forall a\, \Phi \mid \exists a\, \Phi$$

- **Atomic formula:**
  - k-ary Predicate Symbols (k ∈ [1, 2, 3])
  - Variables
    - Unary: P(x)
    - Binary: Q(x, y)
    - Ternary: R(x, y, z)
- **Quantification:**
  - ∀ (Universal), ∃ (Existential)
- **Closed formula:**
  - e.g. √ ∀x P(x) × ∀x P(x, y)
- **Compound formula:**
  - Logical operator: ∧, ∨, →, ¬
  - e.g. ∃xP(x) ∧ ∀y ∀z Q(y, z)

1-3 Quantifier Depth (QD)

- One important measure of the complexity of a FOL formula.

- **For any formula ϕ, ψ**
  - $QD(P(t_{1:m})) = 0$
  - $QD(\phi \wedge \psi) = QD(\phi \vee \psi) = QD(\phi \rightarrow \psi) = \max(QD(\phi), QD(\psi))$
  - $QD(\forall a\phi) = QD(\exists a\phi) = QD(\phi) + 1$
- **Examples**
  - $F_1 = \forall x \exists y \exists z R(x, y, z)$
    - $0 \rightarrow \forall \rightarrow \exists \rightarrow \exists \Rightarrow$ **$QD(F_1) = 3$**
  - $F_2 = \exists x P(x) \wedge \forall y \forall z Q(y, z)$
    - QD (P-side) = 1; QD (Q-side) = 2 $\Rightarrow$ **$QD(F_2) = 2$**
  - $F_3 = \forall x (P(x) \vee \forall y Q(y))$
    - $\forall \rightarrow$ QD (P-side) = 0; QD (Q-side) = 1 $\Rightarrow$ **$QD(F_3) = 2$**

Overview

2-1 Motivation

{a, b}*: ab, bba
Dyck-1: (( ) ( )) ( )

- Recent works[1][2][3] have demonstrated Transformers can recognize different formal languages, such as $a^n b^n$ and Dyck-k. However, such studies do not adequately reflect the real-world texts, where models must capture the structure amid lexical elements.
- To test Transformers' structural sensitivity, we provide an ideal and compact prototype by regarding FOL formulas as a combination of {a, b}* (a =" ∀ ", b =" ∃ "), Dyck-1 and letters. Predicates (uppercase letters) and variables (lowercase letters) introduce a minimal lexical environment; Other parts fix the syntactic structures.
- Through nesting quantifiers in different scopes, QD can reflect the logical depth of FOL. It also highlights how to quantify the contribution of function words in different clauses and composing them over hierarchical phrase structure.

[1] Bhattamishra Satwik, Ahuja Kabir and Goyal Navin. (2020). On the ability and limitations of transformers to recognize formal languages. *Proceedings of the 2020 Conference on Empirical Methods in Natural Language Processing*, pages 7096–7116. Association for Computational Linguistics.

[2] Javid Ebrahimi, Dhruv Gelda, and Wei Zhang (2020). How Can Self-Attention Networks Recognize Dyck-n Languages? *Findings of the Association for Computational Linguistics: EMNLP 2020*. Association for Computational Linguistics, pp. 4301–4306.

[3] Shunyu Yao, Binghui Peng, Christos Papadimitriou, and Karthik Narasimhan (2021). Self-Attention Networks Can Process Bounded Hierarchical Languages. *Proceedings of the 59th Annual Meeting of the Association for Computational Linguistics and the 11th International Joint Conference on Natural Language Processing* (Volume 1: Long Papers). Association for Computational Linguistics, pp. 3770–3785.

Overview

2-1 Motivation

- Recent works have focused on the inner workings and intermediate representations of self-attention.
  - In [4], Weiss et al. create RASP, a symbolic programming lanaguage that formalises the computation in a Transformer encoder as abstract operations on sequences.
  - The most popular approach is visualising attention weights[5][6][7].

- Innovatively, our work integrates RASP algorithms[4], attention analysis, and value-space probing to provide systematic interpretations.

[4] Gail Weiss, Yoav Goldberg, and Eran Yahav (2021). 'Thinking Like Transformers'. In *Proceedings of the 38th International Conference on Machine Learning*. Vol. 139. PMLR, pp. 11080–11090.

[5] Kevin Clark, Urvashi Khandelwal, Omer Levy, and Christopher D. Manning (2019). What Does BERT Look at? An Analysis of BERT's Attention. In *Proceedings of the 2019 ACL Workshop BlackboxNLP: Analyzing and Interpreting Neural Networks for NLP*. Association for Computational Linguistics, pp. 276–286.

[6] Elena Voita, David Talbot, Fedor Moiseev, Rico Sennrich, and Ivan Titov (2019). Analyzing Multi-Head Self-Attention: Specialized Heads Do the Heavy Lifting, the Rest Can Be Pruned. In *Proceedings of the 57th Annual Meeting of the Association for Computational Linguistics*. Association for Computational Linguistics, pp. 5797–5808.

[7] Samira Abnar and Willem Zuidema (2020). Quantifying Attention Flow in Transformers. In *Proceedings of the 58th Annual Meeting of the Association for Computational Linguistics*. Association for Computational Linguistics, pp. 4190–4197.

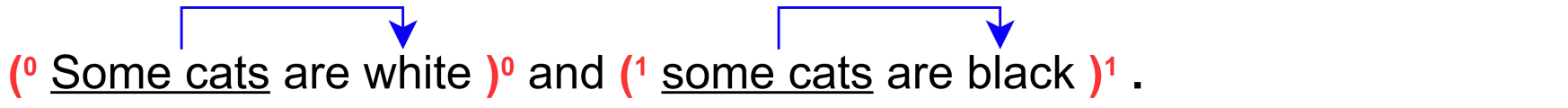
2-2 FOL Types

FOL Type	Example	QD
Standard-1	$\exists x \forall y \forall z P(x, y, z)$	3
Standard-2	$\forall x P(x) \wedge \exists y \forall z R(y, z)$	2
Standard-3	$\forall x P(x) \wedge \exists y \forall z R(y, z) \rightarrow \forall w P(w)$	2
Nested-1	$\forall x (P(x) \vee \forall y Q(y))$	2
Nested-2 (Pattern 1)	$\forall x (P(x) \rightarrow Q(x))$	1
Nested-2 (Pattern 2)	$\exists x (P(x) \wedge \exists y (Q(y) \wedge R(x, y)))$	2
Nested-2 (Pattern 3)	$\exists x (P(x) \wedge \forall (Q(y) \rightarrow \forall z (S(z) \rightarrow R(x, y, z))))$	3

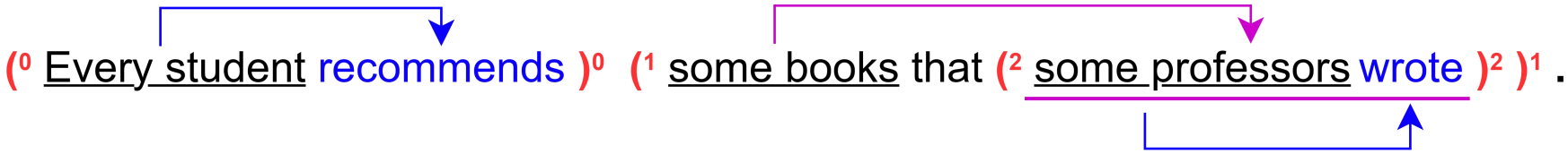
Table 1. Examples of different types of FOL formulas discussed in this study.

2-2 FOL Types

- **Standard**
  - $\exists x P(x) \wedge \exists y Q(y)$
  - [Q] [Noun] [V] [ADJ] [CONJ] [Q] [Noun] [V] [ADJ]



  - $\exists x(P(x) \wedge Q(x)) \wedge \exists y(P(y) \wedge R(x))$
  - $\exists x(Cat(x) \wedge White(x)) \wedge \exists y(Cat(y) \wedge Black(x))$
- **Nested**
  - $\forall x P(x) \rightarrow \exists y(Q(y) \wedge \exists z R(z))$
  - [Q] [Noun] [V] [Q] [Noun] [C] [Q] [Noun] [V]



  - $\forall x(P(x) \rightarrow \exists y(Q(y) \wedge \exists z(S(z) \wedge R(x, y, z) \wedge W(y, z))))$
  - $\forall x(Student(x) \rightarrow \exists y(Professor(y) \wedge \exists z(Book(z) \wedge Recommend(x, y, z) \wedge Write(y, z))))$

Overview

2-3 QD Recognition

- **Task:** Sequence-level classification
- **High-level view of the algorithm**
  - (i) Analyse quantifier scope
  - (ii) Collect the number of quantifiers in each scope
  - (iii) Reset the counts in parallel forms or accumulate them in nested forms
  - (iv) Compare the aggregated information to classify the final result
- **Labels:** QD$\in$[1, 3]

e.g. $\exists x\, P(x) \wedge \forall y \forall z\, Q(y, z)$
$\forall x\, (P(x) \vee \forall y\, Q(y))$

* As natural sentences are rarely observed to exceed three[8][9], our work controls **a maximum QD at 3** to realistically model the hierarchical structure of human languages.

[8] Fred Karlsson (2007). 'Constraints on Multiple Center-Embedding of Clauses'. In *Journal of Linguistics* 43.2, pp. 365–392.

[9] Lifeng Jin, Finale Doshi-Velez, Timothy Miller, William Schuler, and Lane Schwartz (2018). *Depth-bounding is effective: Improvements and evaluation of unsupervised PCFG induction*. In *Proceedings of the 2018 Conference on Empirical Methods in Natural Language Processing*. Association for Computational Linguistics, pp. 2721–2731.

3-1 RASP: The Restricted Access Sequence Processing Language[4]

- Testable hypotheses for white-box explanations
  - Give solutions to tasks that a Transformer encoder can learn
  - Show upper bounds on the required number of layers and heads through simulated self-attention visualisations

- Any algorithm expressible in RASP can be realised in actual Transformers, potentially using a simpler configuration
  - Maintain a counter to accumulate quantifiers
  - Take the maximum QD
  - Perform a reset-like operation at each logical operator when processing parallel subformulas

- In principle, RASP algorithms can handle arbitrary QD.

[4] Gail Weiss, Yoav Goldberg and Eran Yahav. (2021). Thinking like transformers. *International Conference on Machine Learning (PMLR) 2021*, pages 11080-11090.

3-2 RASP ↔ Transfromer Encoder

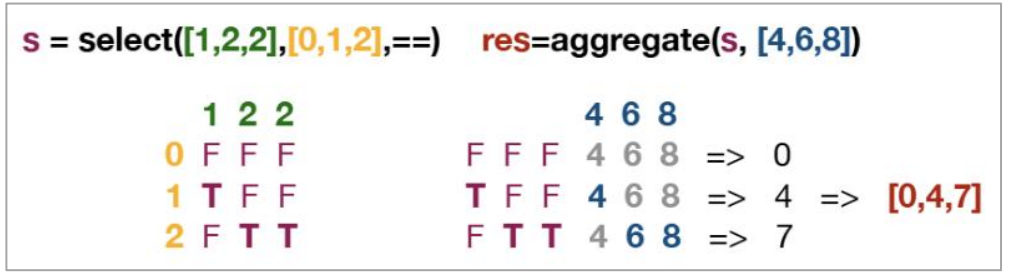
- **Embeddings:**
  - Built-in Sequence Operators (S-op)
  - **tokens:** User-provided tokens of length n ↔ **Token embeddings**
    - **tokens**("hey") = "hey"
  - **indices:** Corresponding integer indices from 0 to n-1 ↔ **Positional encodings**
    - **indices**("hey") = [0,1,2]
  - **length:** Sequence length n
    - **length**("hey") = [3,3,3]
- **Feed-forward networks:**
  - **Elementwise s-ops** ↔ **FFN sublayers**
    - Arithmetic (+, -, *)
      - (**indices***3)("hey") = [0 ,6 ,9] (**tokens**+"a")("hey") = "haeaya"
    - Logical comparsion (<, <=, =, >, >=)
      - ((**indices**+1)==**length**)("hi") = [F, T]
    - Ternary operators (if p then a else b)
      - (**tokens** if (**indices**%2==0) else "-")("hello") = "h-l-o"

3-2 RASP ↔ Transfromer Encoder

- **Self-attention:**
- Mix & combine information from different positions
  - **Select(k, q, p)** ↔ **Attention matrix**
    - Create selection matrices
    - Binary decision of two sequences
      - Boolean predicate **p(k,q)**
        - Select each pair of positions **(i, j)**
      - A **n×n** selector (Boolean matrix)
        - $Sel_{i,j} = p(k_i, q_j)$

- select(indices,indices,<)("hey") = $\begin{bmatrix} F & F & F \\ \mathbf{T} & F & F \\ \mathbf{T} & \mathbf{T} & F \end{bmatrix}$
- select(tokens,tokens,==)("eek") = $\begin{bmatrix} \mathbf{T} & \mathbf{T} & F \\ \mathbf{T} & \mathbf{T} & F \\ F & F & \mathbf{T} \end{bmatrix}$

  - **Aggregate(Sel, v)** ↔ **Hard attention**
    - Simplify attention scores
      - Apply value broadcasting to a selector with a numeric sequence **v**
      - Aggregate and average the values of selected positions and assign 0 to all other positions



3-3 Other Operations

- **selector_width(Sel):**
  - Produce per-position token counts
  - i.e. The frequency of each symbol in the input
  - same=select(tokens,tokens,==)
  - selector_width(same)("hello")=[1,1,2,2,1]
- **sort(tokens, tokens):**
  - Sort all values in ascending order
  - Optionally including a BOS token (§)
  - [1,2,2,1,3] → [1,1,2,2,3]
- **indicator(bools):**
  - Turn a sequence of Bollean values into numerical ones
  - Mapping True to 1 and False to 0
  - [T, F, T] → [1,0,1]

Expressiveness Results

4-0-1 Preambles

Preamble 1: Basic S-ops.

quanti = [“∀”, “∃”] ;
LO = [“∧”, “∨”, “→”] ;
is_quanti = *tokens* in quanti ;
is_LO = *tokens* in LO ;

opens = (*tokens* == “(”) ;
closes = (*tokens* == “)”) ;
bos = (*tokens* == “§”) ;

Preamble 2: Basic Positional Selectors.

select_all = **select**(1, 1, ==) ;
leftward = **select**(*indices*, *indices*, <=) ;
rightward = **select**(*indices*, *indices*, >=) ;
equal = **select**(*indices*, *indices*, ==) ;

Preamble 3: Key/Query-specific Selectors.

k_quanti = **select**(is_quanti, *True*, ==) ;
q_quanti = **select**(*True*, is_quanti, ==) ;
k_LO = **select**(is_LO, *True*, ==) ;
q_LO = **select**(*True*, is_LO, ==) ;
k_bos = **select**(bos, *True*, ==) ;
q_bos = **select**(*True*, bos, ==) ;

select_quanti = k_quanti **and** q_quanti ;
select_LO = k_LO **and** q_LO ;

*All algorithms are built on the assumption that the encoder centers its computations around positional information.

Expressiveness Results

4-0-2 Notations

- **S** A sequence of input token $s_0, s_1, ..., s_{n-1}$
- **$i, j \in [0, n-1]$** Valid positions for the keys and queries
- **$M_{i,j} \in \{0, 1\}^{n \times n}$** The selection matrix in each pair of positions (i, j)
- **$I(\cdot) \in \{0, 1\}$** The indicator function
- **a_j** Uniform average of the value vector on the selected keys at each query position j
- **D_i** The depth of input tokens layered by parentheses
- **D_{max}'** The maximum adjusted depth
- **ℓ** Layer $\in [0, D_{max}' - 1]$
- **Q** Layer-wise counts: $num[\ell_{quanti}]$
- **QD_x ($x \in [1, 4]$)** Quantifier depth of different FOL types
- **sub_QD** Quantifier depth in each subformula

*All algorithms are built on the assumption that the encoder centers its computations around positional information.

Expressiveness Results

4-1 Algorithms for Standard Type

FOL Type	Example	QD
Standard-1	$\exists x \forall y \forall z P(x, y, z)$	3
Standard-2	$\forall x P(x) \wedge \exists y \forall z R(y, z)$	2
Standard-3	$\forall x P(x) \wedge \exists y \forall z R(y, z) \rightarrow \forall w P(w)$	2
Nested-1	$\forall x (P(x) \vee \forall y Q(y))$	2
Nested-2 (Pattern 1)	$\forall x (P(x) \rightarrow Q(x))$	1
Nested-2 (Pattern 2)	$\exists x (P(x) \wedge \exists y (Q(y) \wedge R(x, y)))$	2
Nested-2 (Pattern 3)	$\exists x (P(x) \wedge \forall (Q(y) \rightarrow \forall z (S(z) \rightarrow R(x, y, z))))$	3

Table 1. Examples of different types of FOL formulas discussed in this study.

*In Standard FOL, we define each parallel subformula as a clause.

*All Standard types have a variant algorithm with the prefix “§” acting as the BOS token.

Expressiveness Results

4-1-1 Standard-1

e.g. ∀x P(x)

Algorithm 1-1: Standard-1.

```
def type1_fn( ){
    num_quanti = aggregate(leftward,
                           indicator(is_quanti)) ;
    QD_type1 = (indices + 1) * num_quanti ;
    return QD_type1[-1] ;
}
```

$$M_{i,j} = I(i \leq j)$$

$$v_i = I(is\_quanti(s_i))$$

$$v = (v_i)_{i=0}^{n-1} = (v_0, v_1, \ldots, v_{n-1})^T$$

$$a_j = \frac{\sum_{i=0}^{n-1} M_{i,j} v_i}{\sum_{i=0}^{n-1} M_{i,j}} = \frac{\sum_{i=0}^{j} v_i}{j+1}$$

$$sub\_QD_j = (j+1)a_j$$

$$QD_1 = sub\_QD_{n-1} = na_{n-1}$$

Expressiveness Results

4-1-2 Standard-1 with BOS

e.g. § ∀ x P(x)

Algorithm 1-2: Standard-1 with <bos>.

```
def type1_bos_fn( ){
    QD_type1 = aggregate(k_quanti
          and q_bos, indices) ;
   return QD_type1[0] ;
}
```

$$M_{i,j} = k_{quanti}(s_i) \wedge q_{bos}(s_j)$$

$$a_j = \frac{\sum_{i=0}^{n-1} M_{i,j} \cdot i}{\sum_{i=0}^{n-1} M_{i,j}}$$

$$QD_1 = a_0$$

Expressiveness Results

4-1-3 Standard-2

e.g. $\forall x\, P(x) \wedge \exists y \forall z\, Q(y, z)$

Algorithm 2-1: Standard-2.

```
def type2_fn( ){
    LO_pos = aggregate(select_LO, indices) ;
    LO_idx = aggregate(select_all, LO_pos) * length ;
    left = k_quanti and select(indices, LO_idx, <) ;
    right = k_quanti and select(indices, LO_idx, >) ;
    left_QD = selector_width(left) ;
    right_QD = selector_width(right) ;
    QD_type2 = left_QD if (left_QD > right_QD)
                          else right_QD ;
    return QD_type2 ;
}
```

$$M_{i,j} = I(is\_LO(s_i)) \wedge I(is\_LO(s_j))$$

$$\delta = a_j = \frac{\sum_{i=0}^{n-1} M_{i,j} \cdot i}{\sum_{i=0}^{n-1} M_{i,j}}$$

$$v_\delta = (\delta, \delta, \ldots, \delta)^T$$

$$M^L_{i,j} = k_{quanti}(s_i) \wedge I(i < \delta)$$

$$M^R_{i,j} = k_{quanti}(s_i) \wedge I(i > \delta)$$

$$sub\_QD^L = \sum_{i=0}^{\delta-1} M^L_{i,j}, \quad sub\_QD^R = \sum_{i=\delta+1}^{n-1} M^R_{i,j}$$

$$QD_2 = \mathsf{max}(sub\_QD^L, sub\_QD^R)$$

Expressiveness Results

4-1-4 Standard-2 with BOS

e.g. § ∀ x P(x) ∧ ∃ y ∀ z Q(y, z)

Algorithm 2-2: Standard-2 with <bos>.

```
def type2_bos_fn( ){
    bos_quanti = k_quanti and q_bos ;
    bos_LO = k_LO and q_bos ;
    LO_idx = aggregate(bos_LO, indices) ;
    left = k_quanti and select(indices, LO_idx, <) ;
    right = bos_quanti and select(indices, LO_idx, >) ;
    left_QD = selector_width(left)[0] ;
    right_QD = selector_width(right)[0] ;
    QD_type2 = left_QD if (left_QD > right_QD)
                          else right_QD ;
    return QD_type2 ;
}
```

$$bos_{quanti}(i,j) = k_{quanti}(s_i) \wedge q_{bos}(s_j)$$

$$bos_{LO}(i,j) = k_{s_i}(i,j) \wedge q_{bos}(s_j)$$

$$\delta = a_j = \frac{\sum_{i=0}^{n-1} bos_{LO}(i,j) \cdot i}{\sum_{i=0}^{n-1} bos_{LO}(i,j)}$$

$$M_{i,j}^{R} = bos_{quanti}(i,j) \wedge I(i > \delta)$$

$$sub\_QD^{L} = \sum_{i} M_{i,0}^{L}, \quad sub\_QD^{R} = \sum_{i} M_{i,0}^{R}$$

Expressiveness Results

4-1-5 Standard-3

e.g. ∀x P(x) ∧ ∃y ∀z Q(y, z) → ∀w P(w)

Algorithm 3-1: Standard-3.

```
def type3_fn( ){
    clause_idx = aggregate(leftward, indicator(is_LO))
                    * (indices + 1) ;
    same_clause = select(clause_idx, clause_idx, ==) ;
    freq = selector_width(same_clause) ;
    quanti_pos = aggregate(same_clause,
                    indicator(is_quanti)) ;
    clause_QD = quanti_pos * freq ;
    QD_type3 = sort(QD_in_clause, QD_in_clause) ;
    return QD_type3[-1] ;
}
```

clause_idx: [0, 0, 0, 1, 1, 1, 2, 2, 2]
relative_idx: [0, 1, 2, 0, 1, 2, 0, 1, 2]

$$M_{i,j} = I(i \leq j)$$

$$a_j = \frac{\sum_{i=0}^{j} I(is\_LO(s_i))}{j+1}$$

$$c_j = a_j(j+1)$$

$$M'_{i,j} = I(c_i = c_j)$$

$$r_j = \sum_{i=0}^{n-1} M'_{i,j}$$

$$a'_j = \frac{\sum_{i=0}^{n-1} M'_{i,j} \cdot I(is\_quanti(s_i))}{\sum_{i=0}^{n-1} M'_{i,j}}$$

$$sub\_QD_j = a'_j r_j$$

$$QD_3 = \arg\max_j sub\_QD_j$$

Expressiveness Results

4-1-6 Standard-3 with BOS

e.g. § ∀ x P(x) ∧ ∃ y ∀ z Q(y, z) → ∀ w P(w)

Algorithm 3-2: Standard-3 with <bos>.

```
def type3_bos_fn( ){
    LO_bos = k_bos and q_LO ;
    LO_pos = aggregate(LO_bos, 1, 0) ;
    quanti_bos = k_bos and q_quanti ;
    quanti_pos = aggregate(quanti_bos, 1, 0) ;
    clause_idx = aggregate(leftward, LO_pos)
                    * (indices + 1) ;
    same_clause = select(clause_idx, clause_idx, ==);
    freq = selector_width(same_clause) ;
    clause_q_pos = aggregate(same_clause, quanti_pos) ;
    clause_QD = clause_q_pos * freq ;
    QD = sort(QD_in_clause, QD_in_clause) ;
    QD_type3 = aggregate(k_bos, QD)[0] ;
    return QD_type3 ;
}
```

$$LO_{bos}(i,j) = k_{bos}(s_i) \wedge q_{LO}(s_j)$$

$$quanti_{bos}(i,j) = k_{bos}(s_i) \wedge q_{quanti}(s_j)$$

$$a_j^{LO} = \frac{\sum_i LO_{bos}(i,j) \cdot 1}{\sum_i LO_{bos}(i,j)}$$

$$a_j^{quanti} = \frac{\sum_i quanti_{bos}(i,j) \cdot 1}{\sum_i quanti_{bos}(i,j)}$$

$$a_j = \frac{\sum_{i=0}^{j} a_j^{LO}}{j+1}$$

$$a'_j = \frac{\sum_{i=0}^{n-1} M'_{i,j} \cdot a_j^{quanti}}{\sum_{i=0}^{n-1} M'_{i,j}} = a_j^{quanti}$$

$$a''_j = \frac{\sum_{i=0}^{n-1} k_{bos}(i,j) \cdot \arg\max_j sub\_QD_j}{\sum_{i=0}^{n-1} k_{bos}(i,j)}$$

$$QD_3 = a''_0$$

Expressiveness Results

4-2 Algorithms for Nested Type

FOL Type	Example	QD
Standard-1	$\exists x \forall y \forall z P(x, y, z)$	3
Standard-2	$\forall x P(x) \wedge \exists y \forall z R(y, z)$	2
Standard-3	$\forall x P(x) \wedge \exists y \forall z R(y, z) \rightarrow \forall w P(w)$	2
Nested-1	$\forall x (P(x) \vee \forall y Q(y))$	2
Nested-2 (Pattern 1)	$\forall x (P(x) \rightarrow Q(x))$	1
Nested-2 (Pattern 2)	$\exists x (P(x) \wedge \exists y (Q(y) \wedge R(x, y)))$	2
Nested-2 (Pattern 3)	$\exists x (P(x) \wedge \forall (Q(y) \rightarrow \forall z (S(z) \rightarrow R(x, y, z))))$	3

Table 1. Examples of different types of FOL formulas discussed in this study.

*In Nested FOL, QD is accumulated across layers, beginning with the innermost sub_QD.

Expressiveness Results

4-2-1 Algorithm for layered depth

e.g. ∀x (P(x) ∨ ∀y Q(y))

Preamble 4: Algorithm for layered depth.

```
def num_prevs(bools){
    num = aggregate(leftward, indicator(bools)) ;
    return (indices + 1) * num ;
}

n_opens = num_prevs(opens) ;
n_closes = num_prevs(closes) ;
depth = n_opens - n_closes ;
adjusted_depth = depth + indicator(closes) ;
```

e.g. ∀x (P (x) ∨ ∀y Q (y))

depth: [0 0 1 1 2 2 1 1 1 1 1 2 2 1 0]

adjusted_depth: [0 0 1 1 2 2 2 1 1 1 1 2 2 2 1]

A stack-like counter

"(" → +1, ")" → -1

$$num[(_i] = \sum_{i \le j} I(s_i = [(])$$

$$num[)_i] = \sum_{i \le j} I(s_i = [)])$$

$$D_i = num[(_i] - num[)_i]$$

$$D'_i = D_i + I(s_i = [)])$$

Expressiveness Results

4-2-2 Nested-1

e.g. $\forall x\,(P(x) \vee \forall y\, Q(y))$

$$M_{i,j} = I(D'_i = \ell)$$
$$\ell_{quanti}(i,j) = k_{quanti}(s_i) \wedge M_{i,j}$$
$$\ell_{LO}(i,j) = k_{LO}(s_i) \wedge M_{i,j}$$
$$Q_\ell = num[\ell_{quanti}] = \sum I(\ell_{quanti})$$
$$num[\ell_{LO}] = \sum I(\ell_{LO})$$

$$sub\_QD = \begin{cases} 0 & , \ell = \emptyset \\ \max(Q^L_\ell, Q^R_\ell) & , num[\ell_{LO}] > 0 \\ Q_\ell & , \text{otherwise.} \end{cases}$$

$$QD_4 = \sum_{\ell=0}^{D'_{max}-1} sub\_QD_\ell$$

*In Nested-1, three layers are used to cover the maximum adjusted depth D_{max} ' = 4; therefore, ℓ = 0, 1, . . . , D_{max}' − 1.

Algorithm 4: Nested-1.

```
def layer_type4(num_layer){
    layer = select(adjusted_depth, num_layer, ==) ;
    layer_quanti = k_quanti and layer ;
    layer_LO = k_LO and layer ;
    has_LO = aggregate(layer_LO, 1, 0) > 0 ;
    LO_idx = aggregate(layer_LO, indices) ;
    left = layer_quanti and select(indices, LO_idx, <) ;
    right = layer_quanti and select(indices, LO_idx, >) ;
    left_QD = selector_width(left) ;
    right_QD = selector_width(right) ;
    no_layer = aggregate(layer,1,0) == 0 ;
    only_QD = selector_width(layer_quanti) ;
    layer_QD = 0 if no_layer
                    else
                (max(left_QD, right_QD) if has_LO
                                    else (only_QD)) ;
    return layer_QD ;
}

def type4_fn( ){
    layer0_QD = layer_type4(0) ;
    layer1_QD = layer_type4(1) ;
    layer2_QD = layer_type4(2) ;
    QD_type4 = layer0_QD + layer1_QD + layer2_QD ;
    return QD_type4 ;
}
```

Data Construction

5-1 Synthetic Datasets

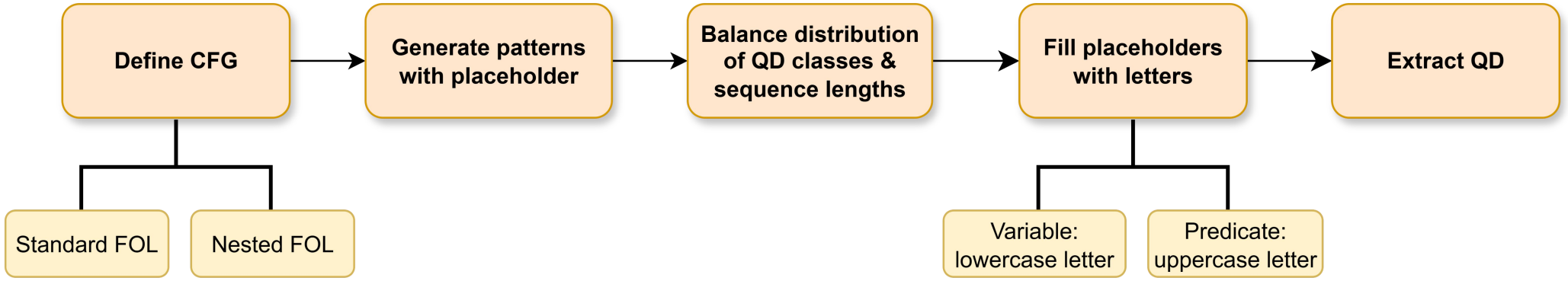
- **Data:**
  - 4,500 samples in each dataset
  - 80% for training, 10% for validation, and 10% for test

- **Pipeline A:** Generate 5 FOL datasets covering all Standard types and Nested − 1.
- **Pipeline B:** Generate 2 FOL2NS datasets, producing three Nested − 2 patterns and their variants combined by logical operators, and mapping them to the corresponding English sentences.

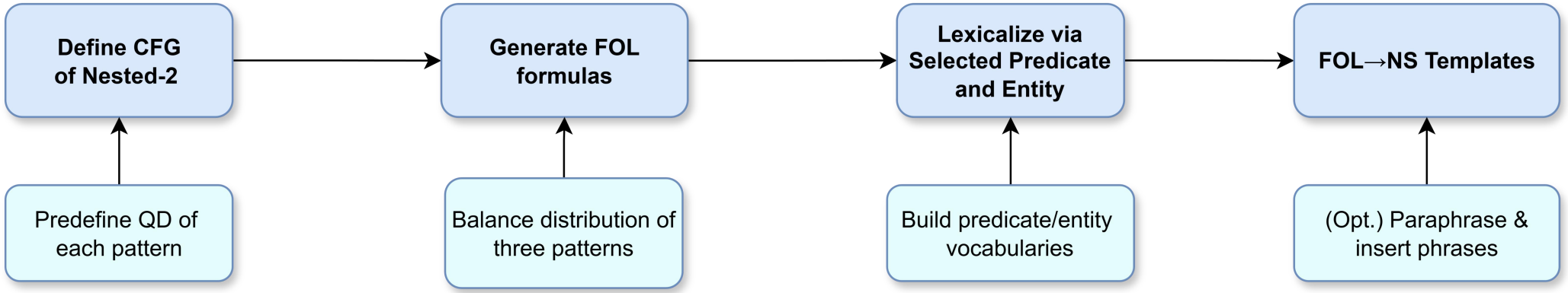
Data Construction

5-1 Synthetic Datasets

Pipeline A: FOL Datasets



Pipeline B: FOL2NS Dataset



Data Construction

5-2-2 FOL2NS Templates

- **Pattern 1:** “[Q] [Person] are [UP].”
- **Pattern 2:** “[Q] [Person] [BP] [Q] [Person].”
- **Pattern 3:** “[Q] [Person] [TP] [Q] [Thing] to [Q] [Person].”

Pattern 1: All/Some {UP_1} are {UP_2}.
$\forall x(P(x) \rightarrow Q(x))$
$\exists x(P(x) \wedge Q(x))$
Pattern 2: All/Some {UP_1} {BP_1} all/some {UP_2}.
$\forall x(P(x) \rightarrow \forall y(Q(y) \rightarrow R(x,y)))$
$\forall x(P(x) \rightarrow \exists y(Q(y) \wedge R(x,y)))$
$\exists x(P(x) \wedge \exists y(Q(y) \wedge R(x,y)))$
$\exists x(P(x) \wedge \forall y(Q(y) \rightarrow R(x,y)))$
Pattern 3: All/Some {UP_1} {TP_1} all/some {UP_2} to all/some {UP_2}.
$\forall x(P(x) \rightarrow \forall(Q(y) \rightarrow \forall z(S(z) \rightarrow R(x,y,z))))$
$\forall x(P(x) \rightarrow \exists(Q(y) \wedge \forall z(S(z) \rightarrow R(x,y,z))))$
$\forall x(P(x) \rightarrow \forall(Q(y) \rightarrow \exists z(S(z) \wedge R(x,y,z))))$
$\forall x(P(x) \rightarrow \exists(Q(y) \wedge \exists z(S(z) \wedge R(x,y,z))))$
$\exists x(P(x) \wedge \exists(Q(y) \wedge \exists z(S(z) \wedge R(x,y,z))))$
$\exists x(P(x) \wedge \forall(Q(y) \rightarrow \exists z(S(z) \wedge R(x,y,z))))$
$\exists x(P(x) \wedge \exists(Q(y) \wedge \forall z(S(z) \rightarrow R(x,y,z))))$
$\exists x(P(x) \wedge \forall(Q(y) \rightarrow \forall z(S(z) \rightarrow R(x,y,z))))$

Table 8. Three patterns and their related English templates in $Nested-2$.

Data Construction

5-2-1 FOL2NS Example

Type	Output
CFG-based FOL pattern	$\exists\_(\#(\_) \wedge \forall\_(\#(\_) \rightarrow \#(\_,\_)))$
FOL Formula	$\exists x(P(x) \wedge \forall y(Q(y) \wedge R(x,y)))$
Lexicalized FOL	$\exists x(Professors(x) \wedge \forall y(Students(y) \wedge Motivate(x,y)))$
Natural Sentence	*Some professors motivate all students.*
Paraphrased Sentence	*It is some professors who motivate all students with good care.*

Table 2. An example of output FOL2NS of $Nested-2$ (Pattern 2) in every step.

Data Construction

5-2-1 FOL2NS Example

Symbol	Lexical Item
Original Formula	$\forall x(P(x) \rightarrow \forall(Q(y) \rightarrow \forall z(S(z) \rightarrow R(x,y,z))))$
Sampling Entity & Predicate	P $\rightarrow$ "$Person_1$": Professors
	Q $\rightarrow$ "$Person_2$": Students
	S $\rightarrow$ "$Thing$": Books
	R $\rightarrow$ "$Ternary\ Predicate$": Explain
Final Output	$\forall x(Professors(x) \rightarrow \forall(Students(y) \rightarrow \forall z(Books(z) \rightarrow Explain(x,y,z))))$

Table 3. An example of the lexicalisation workflow of $Nested-2$ (Pattern 3).

Experimental Setup

6-1 Models

- **Small-sized Transformers** on FOL:
  - Different number of layers (L) and heads (H)
  - Pooling strategies
  - Hidden size: $d_{model} \in \{8, 16, 256\}$
  - Intermediate representation visualisations
    - Mainly on models with **2-layer, 2-head (2L2H)**, **mean pooling** and **d_{model} =256**
    - Attention weights
    - QK score logits (pre-softmax logits)
    - Value vectors (**d_{model} = 8**)

- **BERT-base / 2L2H Transformer** on FOL2NS:
  - 12 layers and 12 heads
  - d_{model} = 758

Experimental Setup

6-2 Training Details

- **Regex-based tokeniser**
- **Custom vocabularies**
  - Necessary special symbols in FOL formulas
  - Uppercase and lowercase letters

    → predicates and variables
  - 480 English words for the FOL2NS vocabulary
- **Performance Metrics**
  - Accuracy
  - Macro-averaged F1
  - F1 scores in each QD class (per-class F1)
- **Others**:
  - Remove all commas to avoid shortcut learning
  - Prepend "§" as BOS marker in some experiments

- **Common configurations:**
  - AdamW optimiser
  - Batch size: 32
  - Learning rate: 1e-5
- **Small-sized Transformers** on FOL:
  - No dropout
  - **Configuration with $d_{model} = 256$**
    - 30 epochs
    - Early stopping with a patience of 10 epoches
  - **Other Configurations**
    - 100 epochs
    - Early stopping with a patience of 15 epoches
- **BERT-base on FOL2NS:**
    - Dropout = 0.2
    - Training for 10 epochs with early stopping at 3 epochs

7-1-1 Performance on FOL

Single-layer:

Underperform in complex structures

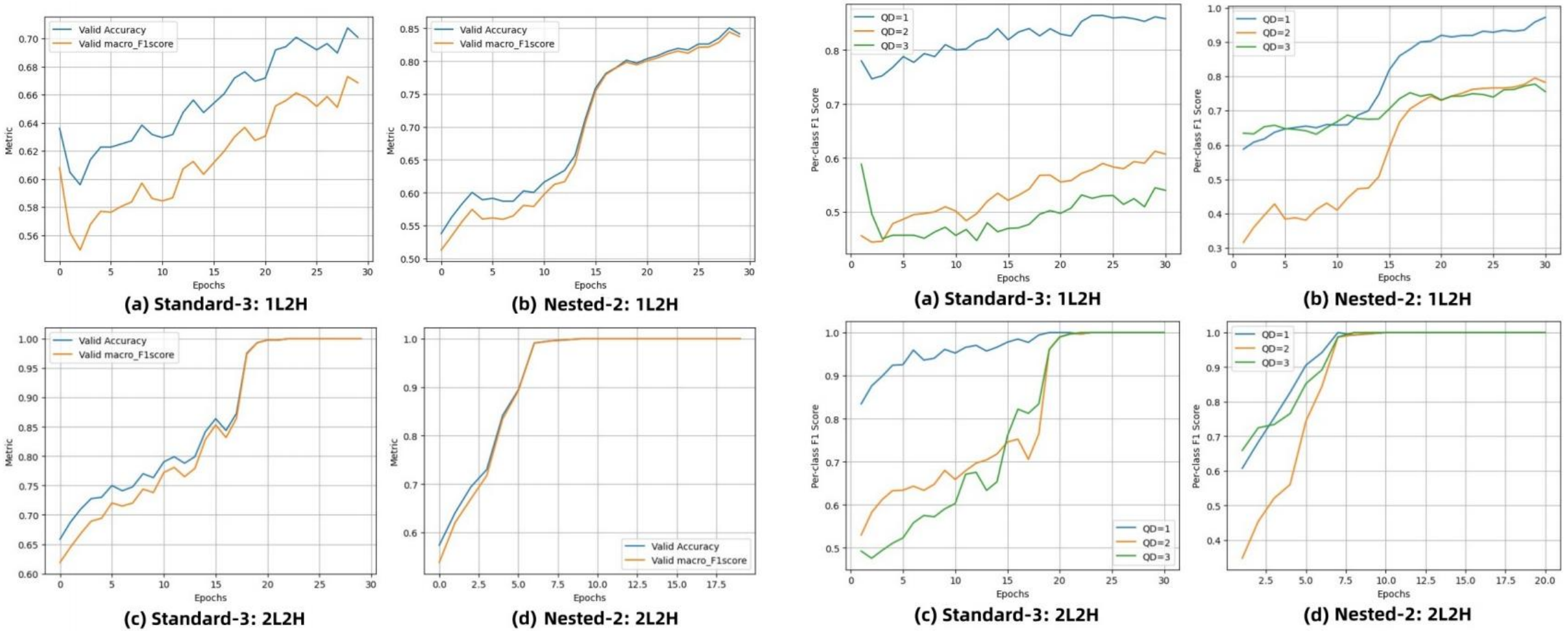
Two-layer:

Handle all FOL types

Layer(L) & Head(H)	Mean Pooling			BOS-token pooling		
	Accuracy	Macro F1	Per-class F1	Accuracy	Macro F1	Per-class F1
Standard-1						
1L1H	100	100	100, 100, 100	74	72	78, 70, 68
1L2H	100	100	100, 100, 100	100	100	100, 100, 100
2L1H	100	100	100, 100, 100	100	100	100, 100, 100
2L2H	100	100	100, 100, 100	100	100	100, 100, 100
Standard-2						
1L2H	81	80	99, 68, 73	83	83	87, 80, 81
2L1H	99	99	99, 100, 100	100	100	100, 100, 100
2L2H	100	100	100, 100, 100	100	100	100, 100, 100
3L2H	100	100	100, 99, 100	100	100	100, 100, 100
Standard-3						
1L2H	48	48	51, 46, 48	70	67	86, 61, 54
2L1H	99	99	99, 100, 100	100	100	100, 100, 100
2L2H	100	100	100, 100, 100	100	100	100, 100, 100
3L2H	100	100	100, 100, 100	100	100	100, 100, 100
Nested-1						
1L1H	100	100	100, 100, 100	82	81	86, 79, 78
1L2H	100	100	100, 100, 100	84	83	91, 74, 83
2L1H	100	100	100, 100, 100	100	100	100, 100, 100
2L2H	100	100	100, 100, 100	100	100	100, 100, 100
Nested-2						
1L2H	84	84	97, 78, 76	48	46	48, 40, 50
1L4H	88	87	93, 82, 85	85	84	91, 77, 85
2L1H	99	99	100, 99, 99	100	100	100, 100, 100
2L2H	99	99	100, 99, 99	100	100	100, 100, 100

Table 5. Under 30 epochs, performance of Transformers with $d_{model} = 256$ across different types of FOL formulas. Per-class: $QD = 1$, $QD = 2$, $QD = 3$.

- Face greater challenges as the structural complexity or QD increases



(a) Accuracy and macro F1. **(b)** Per-class F1.

Figure 2. The validation metrics of a 1L2H Transformer (top) and a 2L2H Transformer (bottom) across epochs on $Stadnard-3$ and $Nested-2$.

- Mean pooling generally works better

Layer(L) & Head(H)	Mean Pooling			BOS-token pooling		
	Accuracy	Macro F1	Per-class F1	Accuracy	Macro F1	Per-class F1
Standard-1						
1L1H	100	100	100, 100, 100	74	72	78, 70, 68
1L2H	100	100	100, 100, 100	100	100	100, 100, 100
2L1H	100	100	100, 100, 100	100	100	100, 100, 100
2L2H	100	100	100, 100, 100	100	100	100, 100, 100
Standard-2						
1L2H	81	80	99, 68, 73	83	83	87, 80, 81
2L1H	99	99	99, 100, 100	100	100	100, 100, 100
2L2H	100	100	100, 100, 100	100	100	100, 100, 100
3L2H	100	100	100, 99, 100	100	100	100, 100, 100
Standard-3						
1L2H	48	48	51, 46, 48	70	67	86, 61, 54
2L1H	99	99	99, 100, 100	100	100	100, 100, 100
2L2H	100	100	100, 100, 100	100	100	100, 100, 100
3L2H	100	100	100, 100, 100	100	100	100, 100, 100
Nested-1						
1L1H	100	100	100, 100, 100	82	81	86, 79, 78
1L2H	100	100	100, 100, 100	84	83	91, 74, 83
2L1H	100	100	100, 100, 100	100	100	100, 100, 100
2L2H	100	100	100, 100, 100	100	100	100, 100, 100
Nested-2						
1L2H	84	84	97, 78, 76	48	46	48, 40, 50
1L4H	88	87	93, 82, 85	85	84	91, 77, 85
2L1H	99	99	100, 99, 99	100	100	100, 100, 100
2L2H	99	99	100, 99, 99	100	100	100, 100, 100

Table 5. Under 30 epochs, performance of Transformers with $d_{model} = 256$ across different types of FOL formulas. Per-class: $QD = 1$, $QD = 2$, $QD = 3$.

- Parallel form is harder than nest form

Layer(L) & Head(H)	Mean Pooling			BOS-token pooling		
	Accuracy	Macro F1	Per-class F1	Accuracy	Macro F1	Per-class F1
Standard-1						
1L1H	100	100	100, 100, 100	74	72	78, 70, 68
1L2H	100	100	100, 100, 100	100	100	100, 100, 100
2L1H	100	100	100, 100, 100	100	100	100, 100, 100
2L2H	100	100	100, 100, 100	100	100	100, 100, 100
Standard-2						
1L2H	81	80	99, 68, 73	83	83	87, 80, 81
2L1H	99	99	99, 100, 100	100	100	100, 100, 100
2L2H	100	100	100, 100, 100	100	100	100, 100, 100
3L2H	100	100	100, 99, 100	100	100	100, 100, 100
Standard-3						
1L2H	48	48	51, 46, 48	70	67	86, 61, 54
2L1H	99	99	99, 100, 100	100	100	100, 100, 100
2L2H	100	100	100, 100, 100	100	100	100, 100, 100
3L2H	100	100	100, 100, 100	100	100	100, 100, 100
Nested-1						
1L1H	100	100	100, 100, 100	82	81	86, 79, 78
1L2H	100	100	100, 100, 100	84	83	91, 74, 83
2L1H	100	100	100, 100, 100	100	100	100, 100, 100
2L2H	100	100	100, 100, 100	100	100	100, 100, 100
Nested-2						
1L2H	84	84	97, 78, 76	48	46	48, 40, 50
1L4H	88	87	93, 82, 85	85	84	91, 77, 85
2L1H	99	99	100, 99, 99	100	100	100, 100, 100
2L2H	99	99	100, 99, 99	100	100	100, 100, 100

Table 5. Under 30 epochs, performance of Transformers with $d_{model} = 256$ across different types of FOL formulas. Per-class: $QD = 1$, $QD = 2$, $QD = 3$.

- Hidden dimension isn't the bottleneck

FOL Type	$d_{model} = 8$			$d_{model} = 16$		
	Accuracy	Macro F1	Per-class F1	Accuracy	Macro F1	Per-class F1
Standard-1	100	100	100, 100, 100	100	100	100, 100, 100
Standard-2	93	92	96, 88, 93	100	100	100, 100, 100
Standard-3	86	85	94, 84, 77	100	100	100, 100, 100
Nested-1	97	97	99, 96, 97	100	100	100, 100, 100
Nested-2	98	98	100, 98, 98	100	100	100, 100, 100

Table 6. Under 100 epochs, performance of 2L2H Transformers with mean pooling strategy across different FOL types. Per-class: $QD = 1$, $QD = 2$, $QD = 3$.

Results

7-1-2 Performance on FOL2NS

- Find stable shortcuts
- Ignore peripheral parts

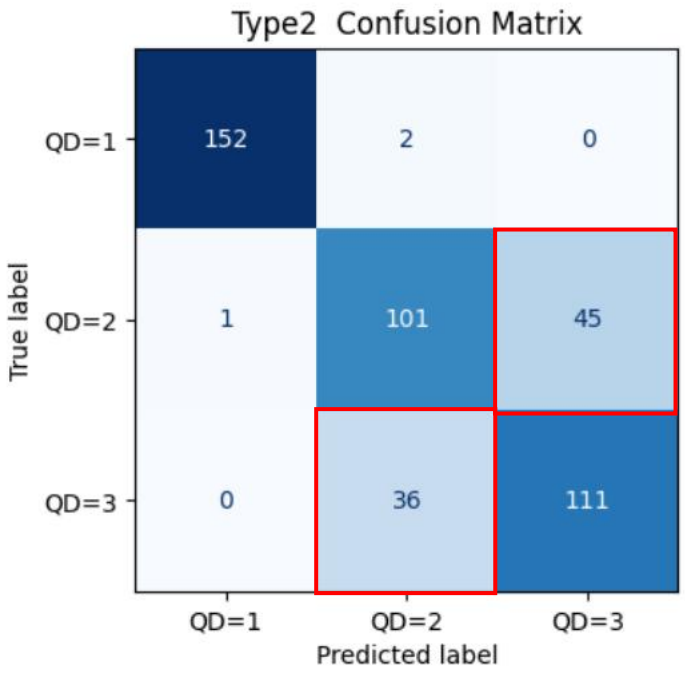
FOL2NS Type	BERT			2L2H Transformer		
	Accuracy	Macro F1	Per-class F1	Accuracy	Macro F1	Per-class F1
Natural Sentence	100	100	100, 100, 100	100	100	100, 100, 100
Paraphrased Sentence	100	100	100, 100, 100	100	100	100, 100, 100

Table 7. Performance of BERT and 2L2H Transformers on two FOL2NS types. Per-class: $QD = 1$, $QD = 2$, $QD = 3$.

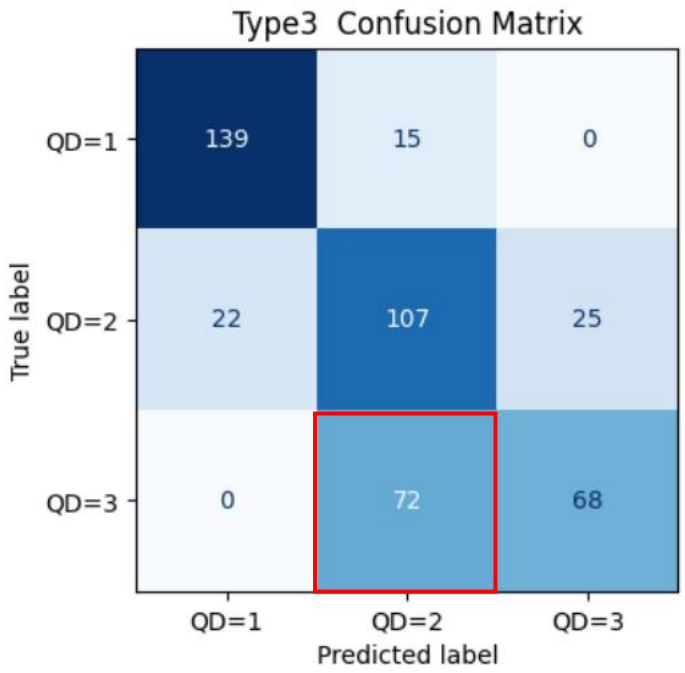
Interpretations

7-2-1 Limitations of One Layer

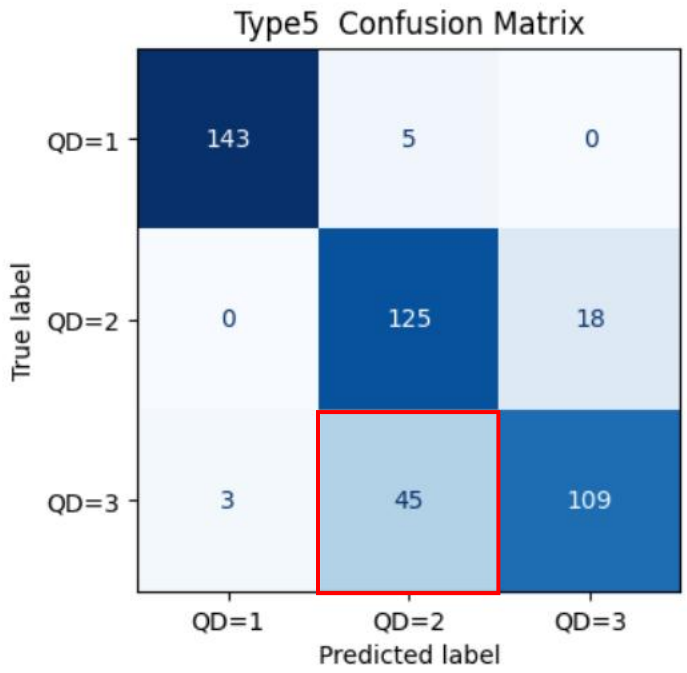
- Struggle with choosing results



(a) $Standard-2.$



(b) $Standard-3.$



(c) $Nested-2.$

*Classifier logits: Top-1 logit (≈0.48) and top-2 logit (≈0.45) present a small margin of only 0.03 across all error cases.

7-2-1 Limitations of One Layer

- Require a reset-like operation at every logical operator

- **Bhattamishra et al. (2020)[1]**
  - Limitations on Reset-Dyck-1 (#);
  - Both attention score <$Q(x_n)$, $K(\cdot)$> and value vector at reset symbols are independent of preceding inputs
  - × Reset computations

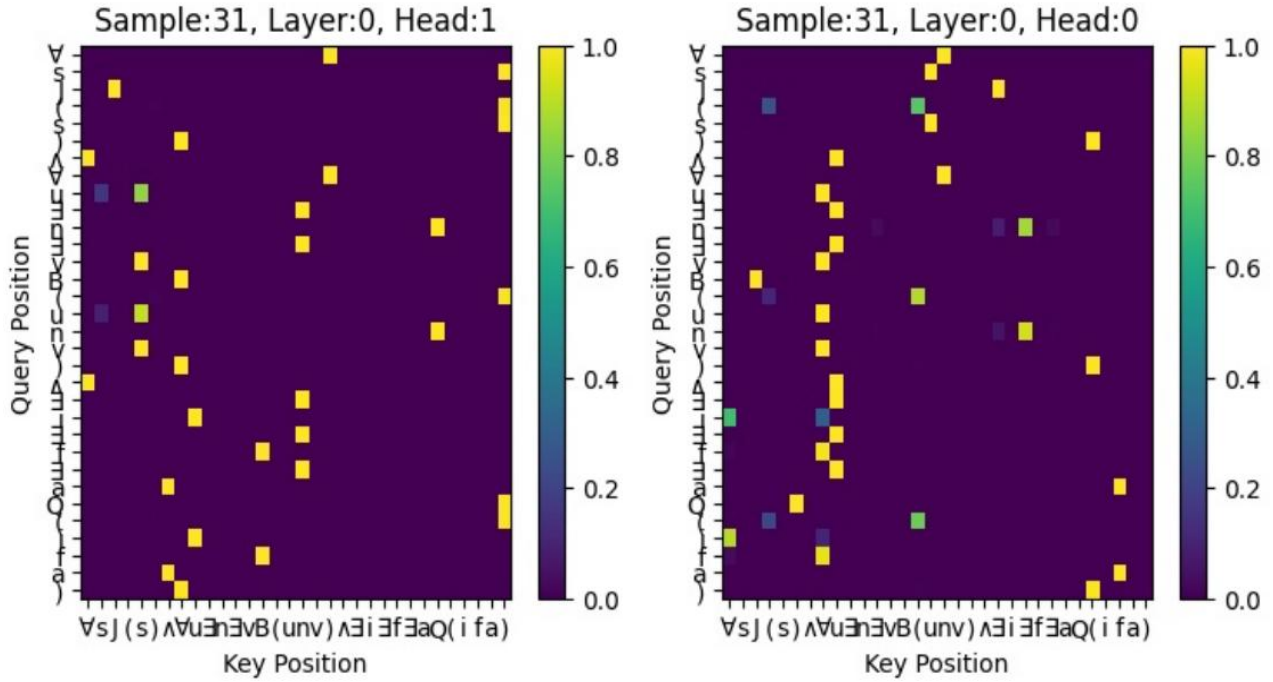
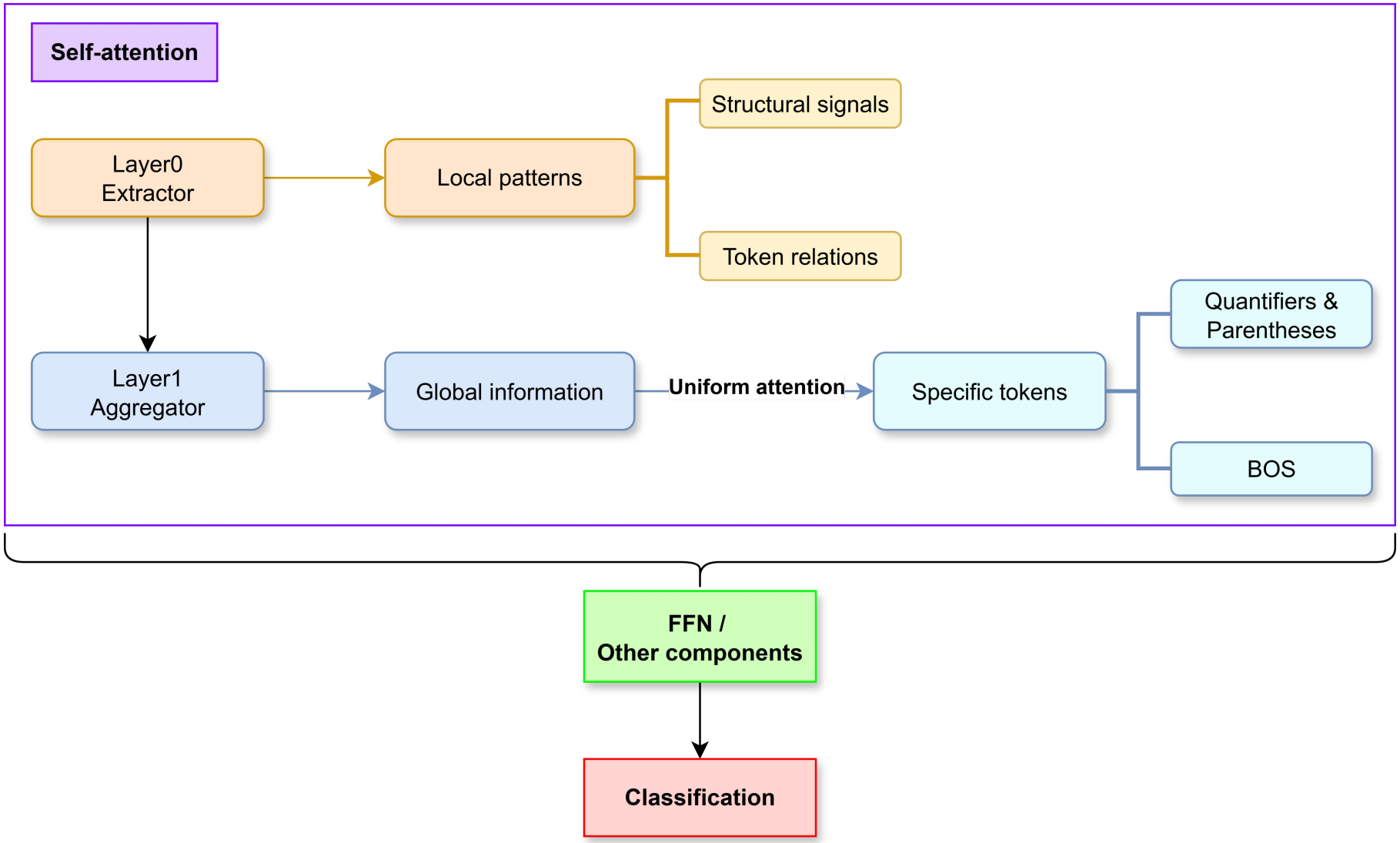


Figure 4. A heatmap example of a 1L2H Transformer on $Standard - 3$ with $QD = 3$.

[1] Bhattamishra Satwik, Ahuja Kabir and Goyal Navin. (2020). On the ability and limitations of transformers to recognize formal languages. *Proceedings of the 2020 Conference on Empirical Methods in Natural Language Processing*, pages 7096–7116. Association for Computational Linguistics.

Interpretations

7-2-2 Roles of Two Layers



- **Merrill et al. (2021)[8]**
  - Further formalise argmax and mean operations under the saturated attention → Count over tokens

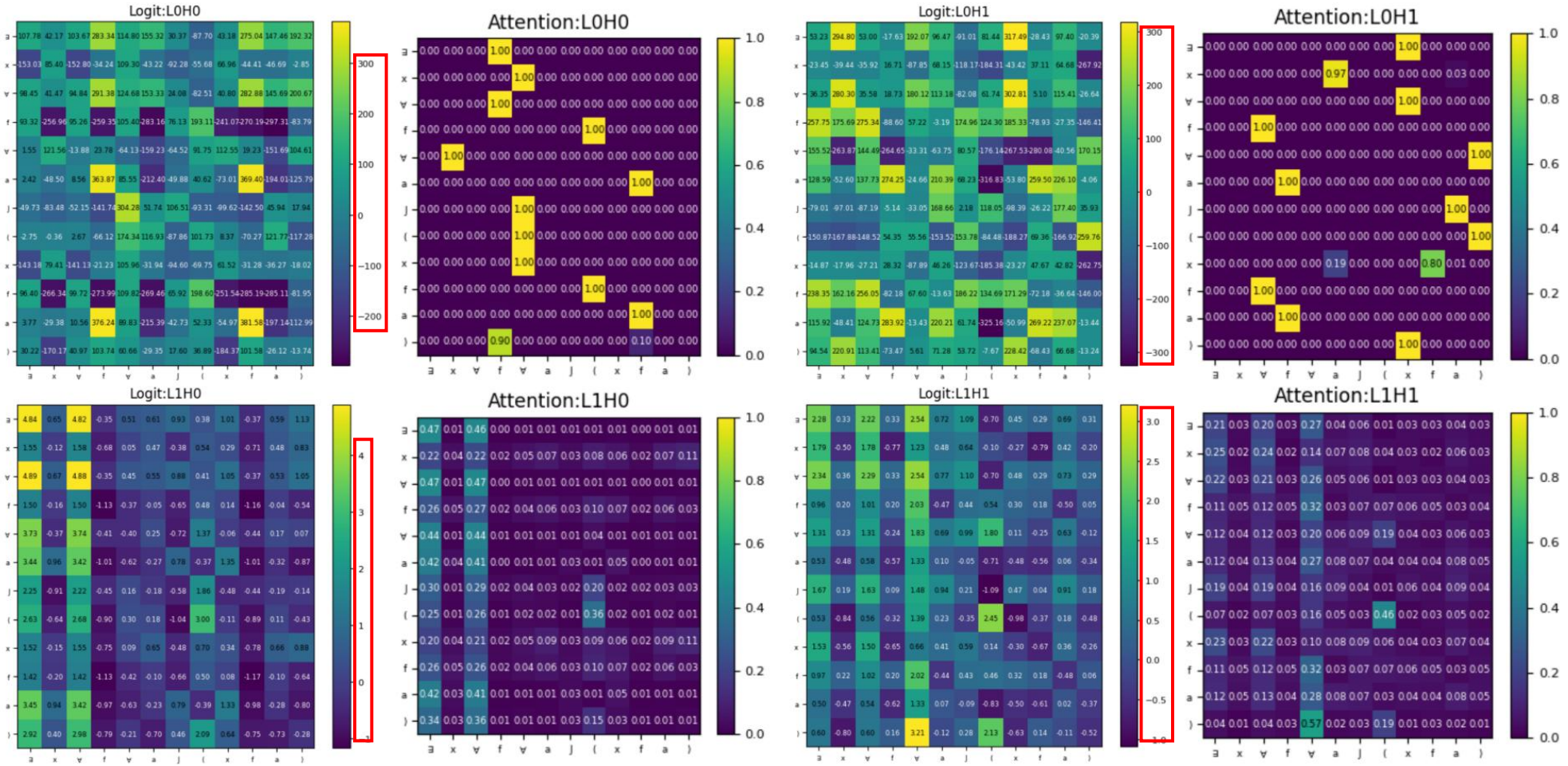


Figure 5. A heatmap comparison between score logits and attention weights of a 2L2H Transformer on $Standard - 1$ with $QD = 3$.

[8] William Merrill, Vivek Ramanujan, Yoav Goldberg, Roy Schwartz, and Noah A. Smith (2021). Effects of Parameter Norm Growth During Transformer Training: Inductive Bias from Gradient Descent. In *Proceedings of the 2021 Conference on Empirical Methods in Natural Language Processing*. Association for Computational Linguistics.

Interpretations

7-2-2 Roles of Two Layers

- **Layer0 → Layer1**
  - Layer0 remains a wide span
  - Layer1 narrows its value range

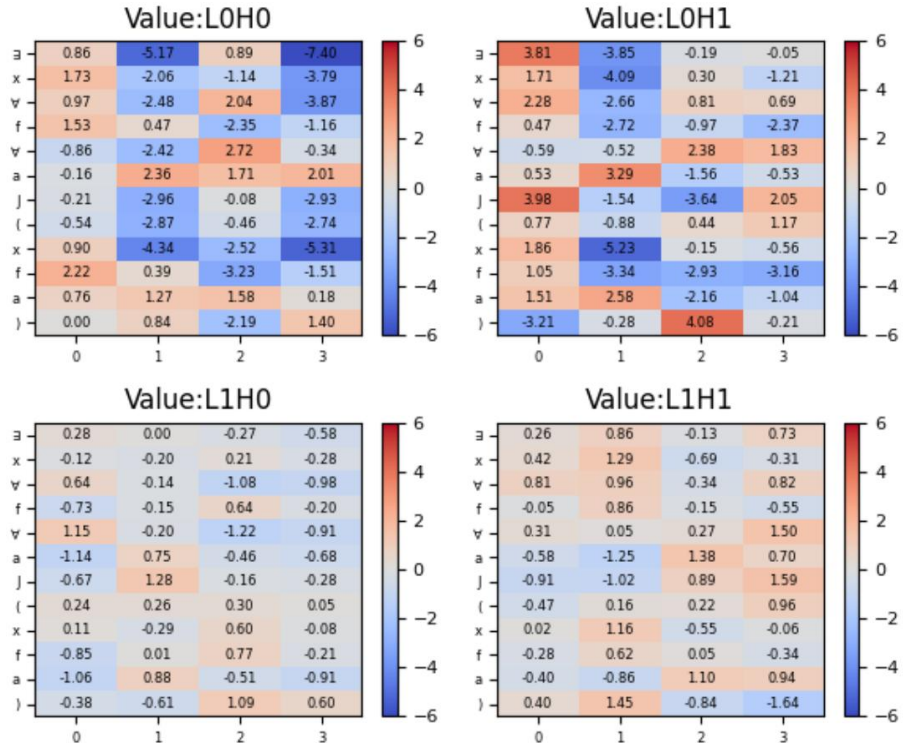


Figure 6. An example of value vector visualisations of a 2L2H Transformer with $d_{model} = 8$ on $Standard - 1$ with $QD = 3$.

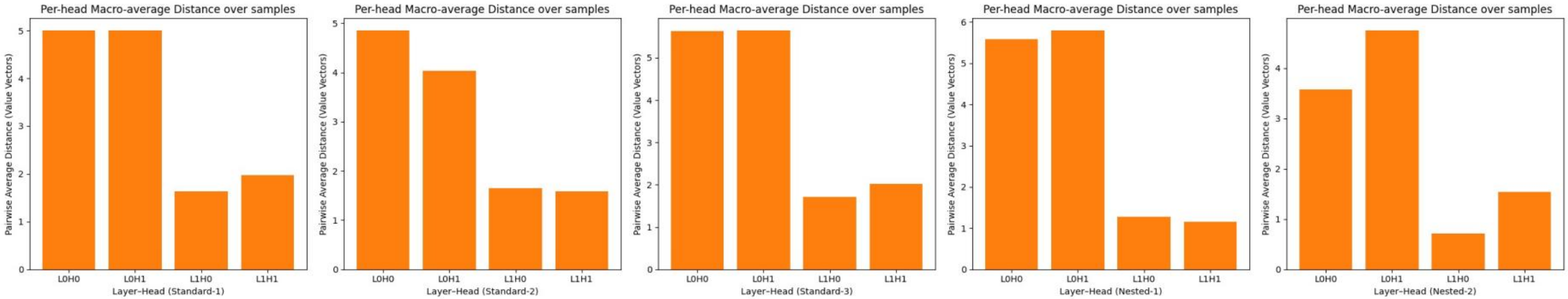
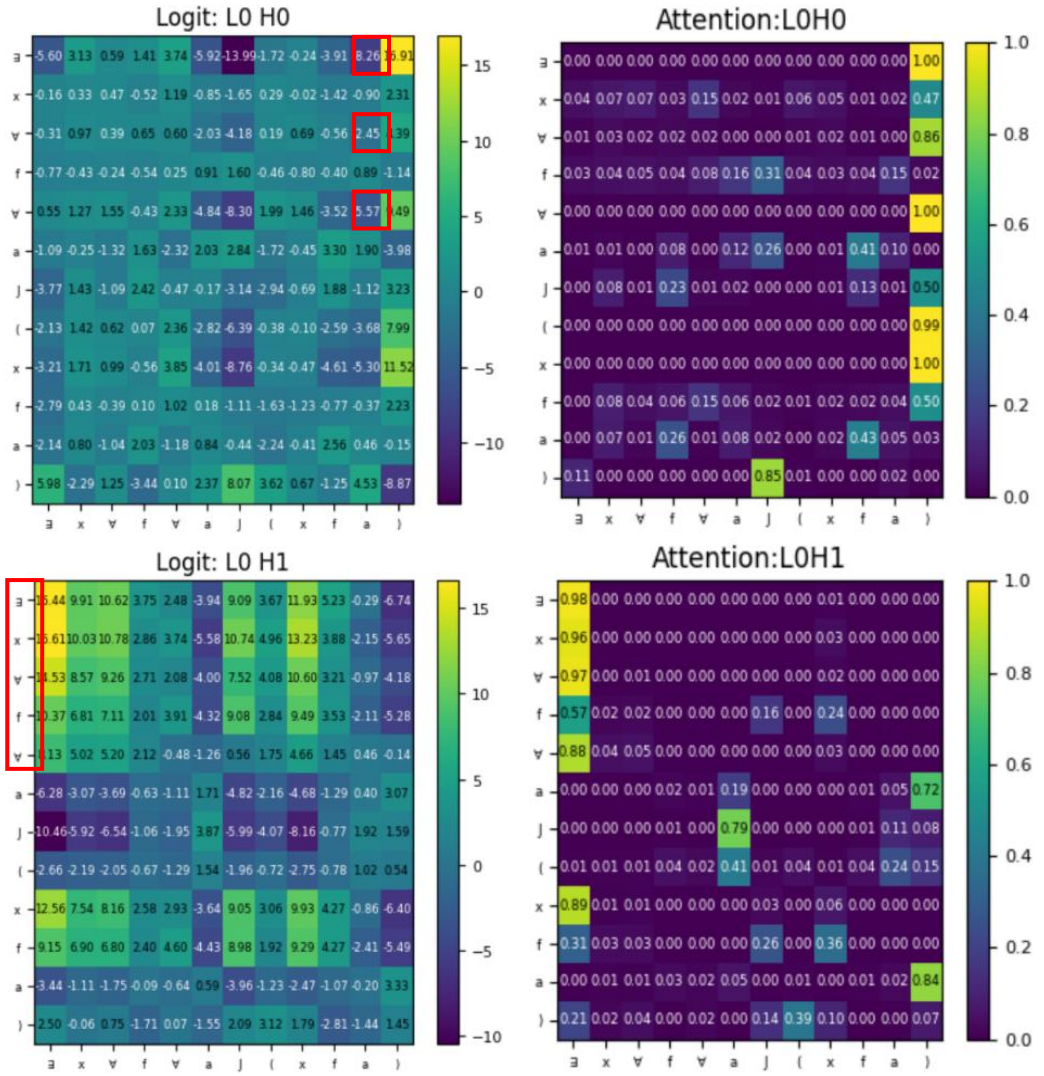


Figure 7. Per-head macro-average distance of value vectors across five FOL types.

Layer0 → “Extractor”

- 1st round of interactions across positions
- Head-level specialisation in Standard − 1 (dmodel = 8)
  - Structural / Relational roles
- **Clark et al. (2019)[5], Michel et al. (2019)[9], Voita et al.[10] (2019)**
  - Characterise attention heads in larger architectures



[5] Kevin Clark, Urvashi Khandelwal, Omer Levy, and Christopher D. Manning (2019). What Does BERT Look at? An Analysis of BERT’s Attention. In *Proceedings of the 2019 ACL Workshop BlackboxNLP: Analyzing and Interpreting Neural Networks for NLP*. Association for Computational Linguistics, pp. 276–286.

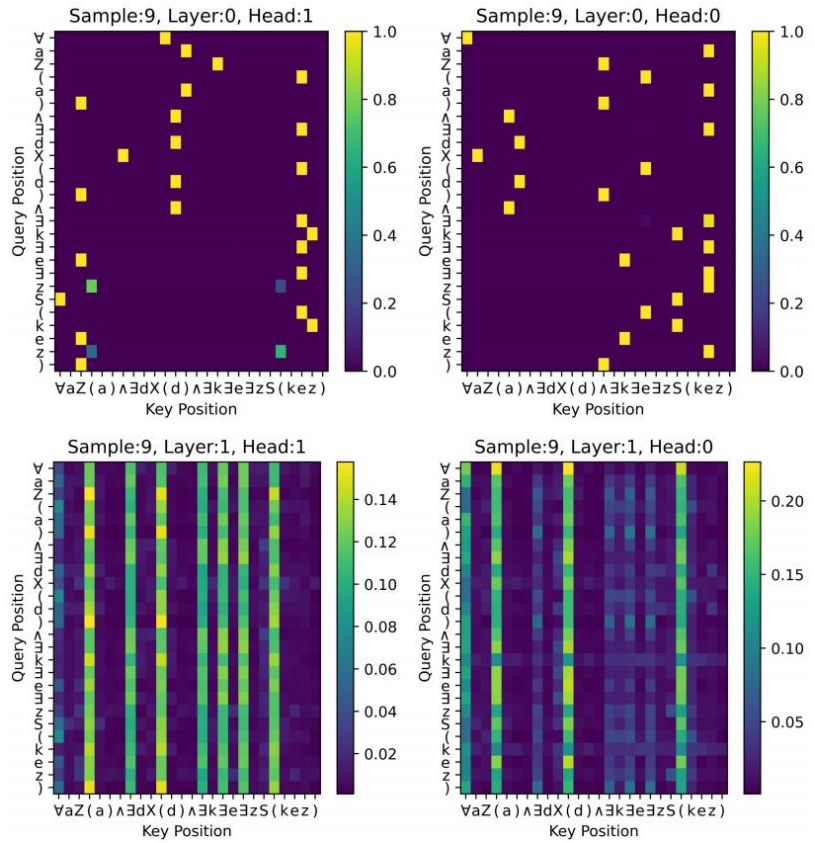
[9] Paul Michel, Omer Levy, and Graham Neubig (2019). Are Sixteen Heads Really Better than One? In *Advances in Neural Information Processing Systems*. Vol. 32. Curran Associates, Inc.

[10] Elena Voita, David Talbot, Fedor Moiseev, Rico Sennrich, and Ivan Titov (2019). Analyzing Multi-Head Self-Attention: Specialized Heads Do the Heavy Lifting, the Rest Can Be Pruned. In *Proceedings of the 57th Annual Meeting of the Association for Computational Linguistics*. Association for Computational Linguistics, pp. 5797–5808.

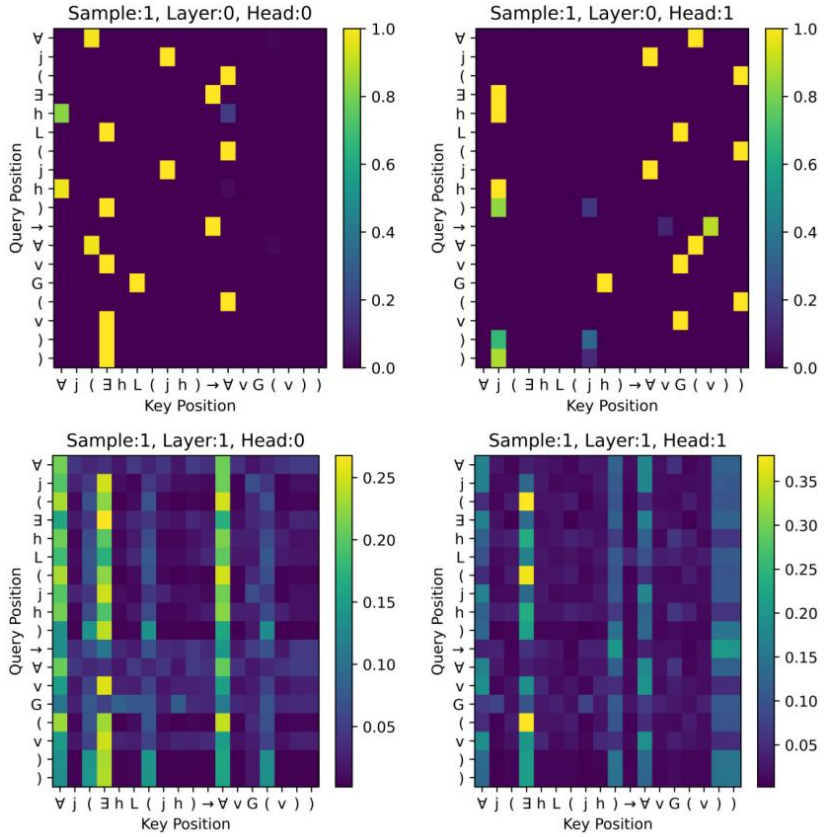
Layer1 → “Aggregator”

(i) Integrate the important features extracted from Layer0 into target positions

- Aggregate structural information with token-level relations
- Distinguishes quantifier scopes by aligning quantifiers with either “(” or “)”



(a) $Standard-3$ with $QD=3$

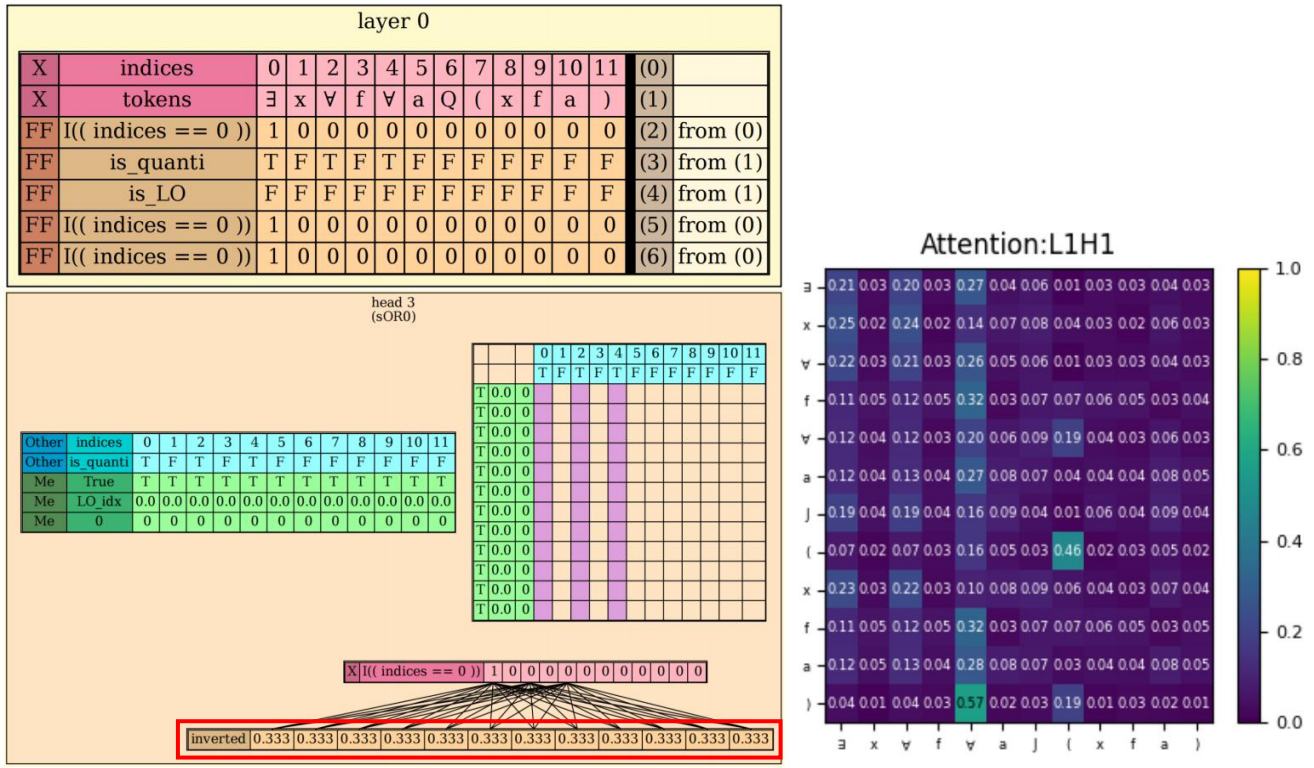


(b) $Nested-1$ with $QD=3$.

Layer1 → “Aggregator”

(ii) Implement a global-average behaviour via the uniform attention

- Average operations → A global readout
- Place nearly uniform attention weights over tied positions



(a) In RASP visualisation. **(b)** In a 2L2H Transformer.

Figure 10. A matching head of $Standard-1$ when processing “$\exists x \forall f \forall a Q(xfa)$”.

Interpretations

7-2-2 Roles of Two Layers

- **Yao et al. (2021)[3], Abnar and Zuidema (2020)[7], Ebrahimi et al. (2020)[11]**
  - Lower layers: position-specific attention patterns
  - Higher layers: tend toward near-uniform weights or virtually hard attention

- **Bhattamishra et al. (2020)[1]**
  - Set Query and Value to the identity matrices
    → A uniform attention score $\frac{1}{i}\sum_{j=1}^{i} v_j$ at position i as the mean of all prefix tokens
  - Dyck-k languages
    - Every token contributes equally to the final result

[1] Bhattamishra Satwik, Ahuja Kabir and Goyal Navin. (2020). On the ability and limitations of transformers to recognize formal languages. *Proceedings of the 2020 Conference on Empirical Methods in Natural Language Processing*, pages 7096–7116. Association for Computational Linguistics.

[3] Shunyu Yao, Binghui Peng, Christos Papadimitriou, and Karthik Narasimhan (2021). Self-Attention Networks Can Process Bounded Hierarchical Languages. *Proceedings of the 59th Annual Meeting of the Association for Computational Linguistics and the 11th International Joint Conference on Natural Language Processing* (Volume 1: Long Papers). Association for Computational Linguistics, pp. 3770–3785.

[7] Samira Abnar and Willem Zuidema (2020). Quantifying Attention Flow in Transformers. In *Proceedings of the 58th Annual Meeting of the Association for Computational Linguistics*. Association for Computational Linguistics, pp. 4190–4197.

[11] Javid Ebrahimi, Dhruv Gelda, and Wei Zhang (2020). How Can Self-Attention Networks Recognize Dyck-n Languages?' In *Findings of the Association for Computational Linguistics: EMNLP 2020*. Association for Computational Linguistics, pp. 4301–4306.

7-2-2 Roles of Two Layers

- **RASP**
  - Self-attention → W_O + FFN
    - Linearly decodable for decision-making
- **Geva et al. (2021)** [12]
  - FFN → Unnormalised key-value memories
  - Residual connections → Store and transmit information
  - Others: Refinements

X	inverted	1	1	1	1	1	1	1	1	1	1	1	1	1	(0)	
X	valat0	1	1	1	1	1	1	1	1	1	1	1	1	1	(1)	
X	inverted	0.5	0.5	0.5	0.5	0.5	0.5	0.5	0.5	0.5	0.5	0.5	0.5	0.5	(2)	
X	valat0	0	0	0	0	0	0	0	0	0	0	0	0	0	(3)	
FF	except0	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0	(4)	from (0)
FF	left_QD	1	1	1	1	1	1	1	1	1	1	1	1	1	(5)	from (1, 4)
FF	except0	1.0	1.0	1.0	1.0	1.0	1.0	1.0	1.0	1.0	1.0	1.0	1.0	1.0	(6)	from (2)
FF	right_QD	1	1	1	1	1	1	1	1	1	1	1	1	1	(7)	from (6, 3)
FF	QD_type2_fn()	1	1	1	1	1	1	1	1	1	1	1	1	1	(8)	from (7, 5)

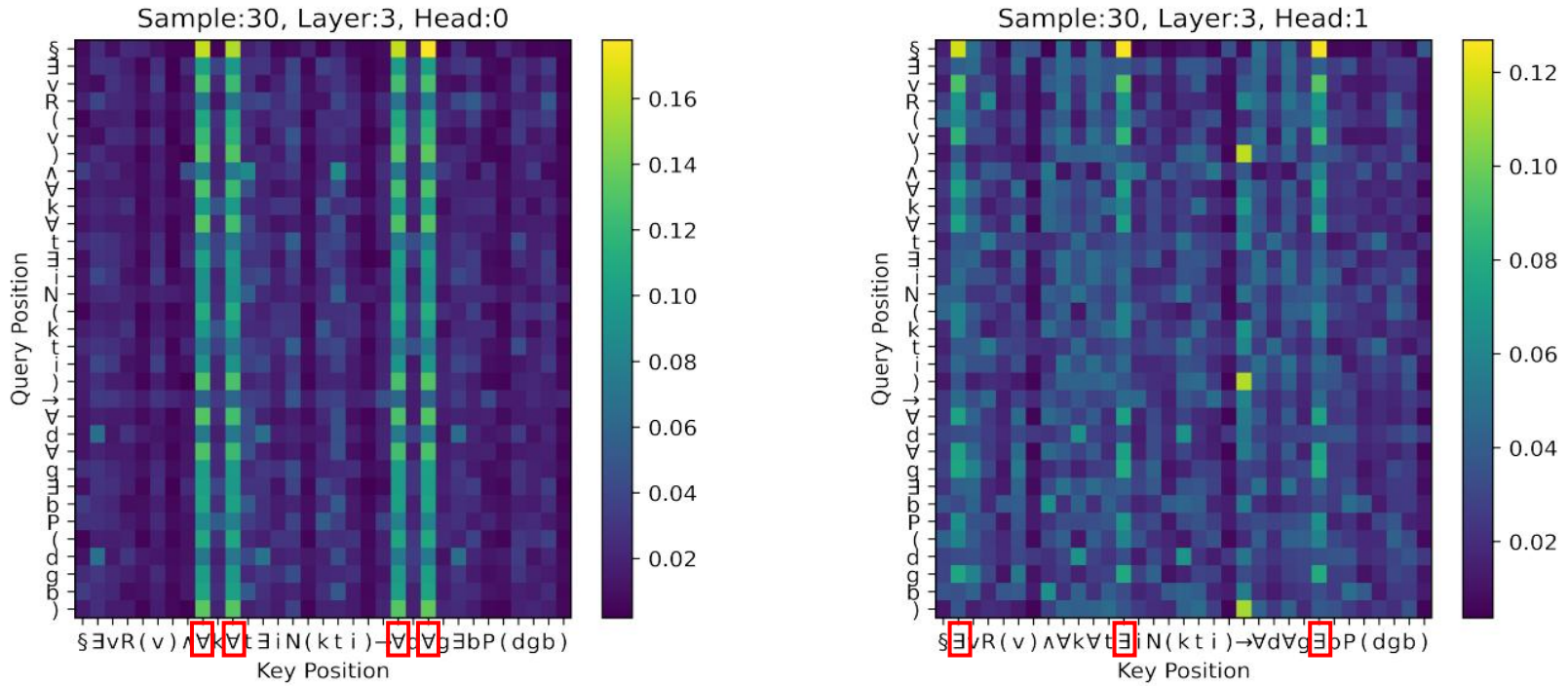
Figure 11. The component that simulates FFN sublayers in the RASP visualisation when processing "$\forall g E(g) \rightarrow \forall b F(b)$".

[12] Mor Geva, Roei Schuster, Jonathan Berant, and Omer Levy (2021). Transformer Feed-Forward Layers Are Key-Value Memories. In *Proceedings of the 2021 Conference on Empirical Methods in Natural Language Processing*. Association for Computational Linguistics, pp. 5484–5495.

Interpretations

7-2-3 Relations between Quantifiers

- Type-specific preferences



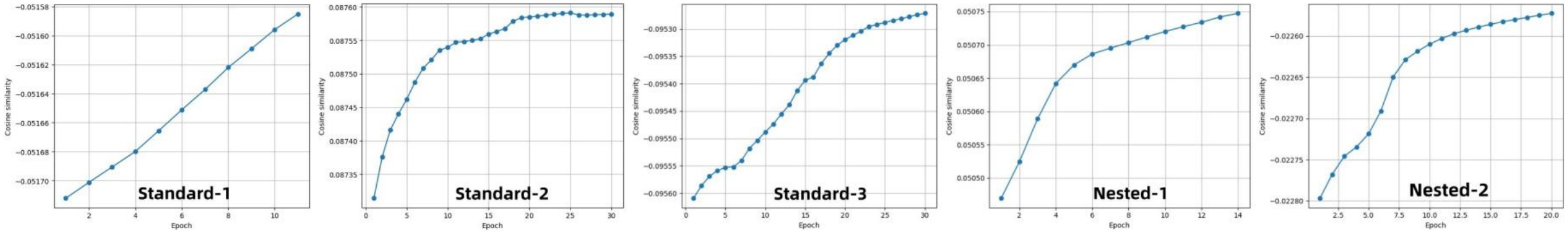
(a) Layer3, Head0. **(b)** Layer3, Head1.

Figure 12. Two heads in Layer3 of a 4L2H Transformer with BOS-token pooling on $Standard - 3$ with $QD = 3$.

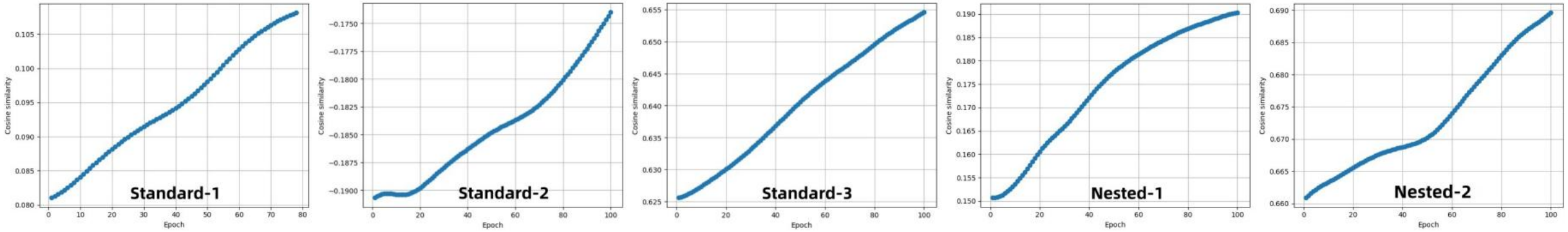
Interpretations

7-2-3 Relations between Quantifiers

- Lexical embeddings throughout training



(a) $d_{model} = 256$ with 30 epochs.



(b) $d_{model} = 8$ with 100 epochs.

Figure 13. The trend of quantifier cosine similarities across all FOL types on a 2L2H Transformer during training.

Interpretations

7-2-3 Relations between Quantifiers

- Functionally similar but not identical
  - Learn relations through contextual representations

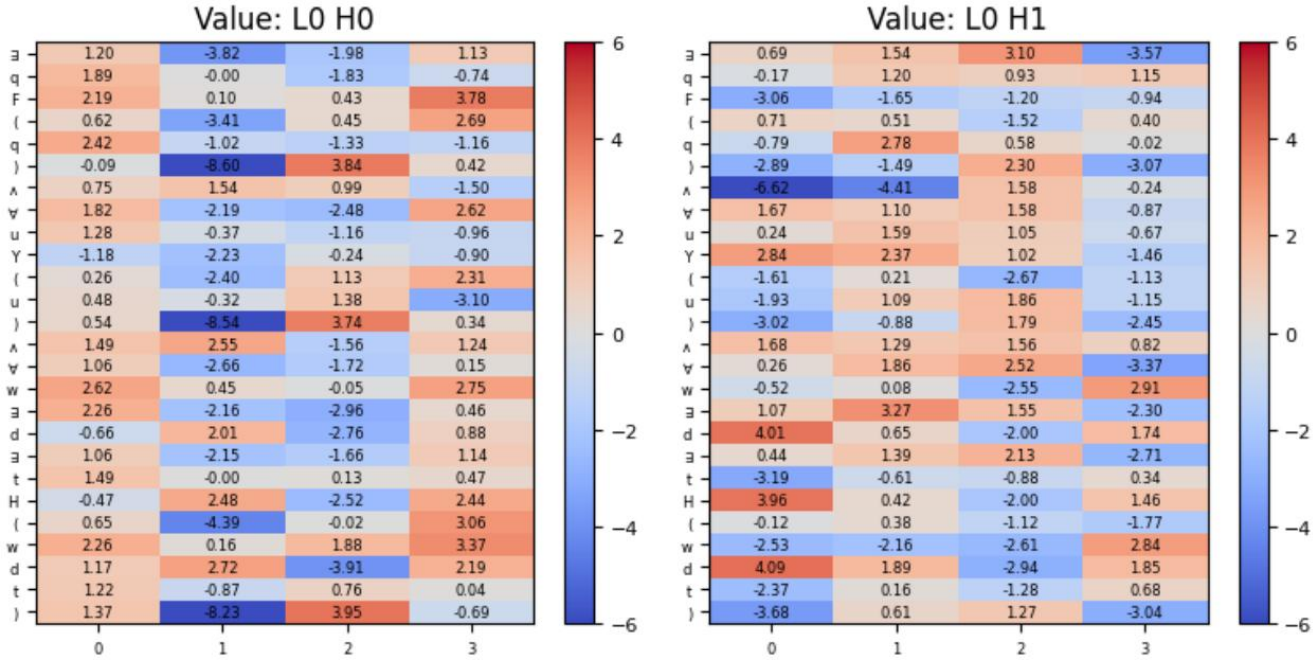
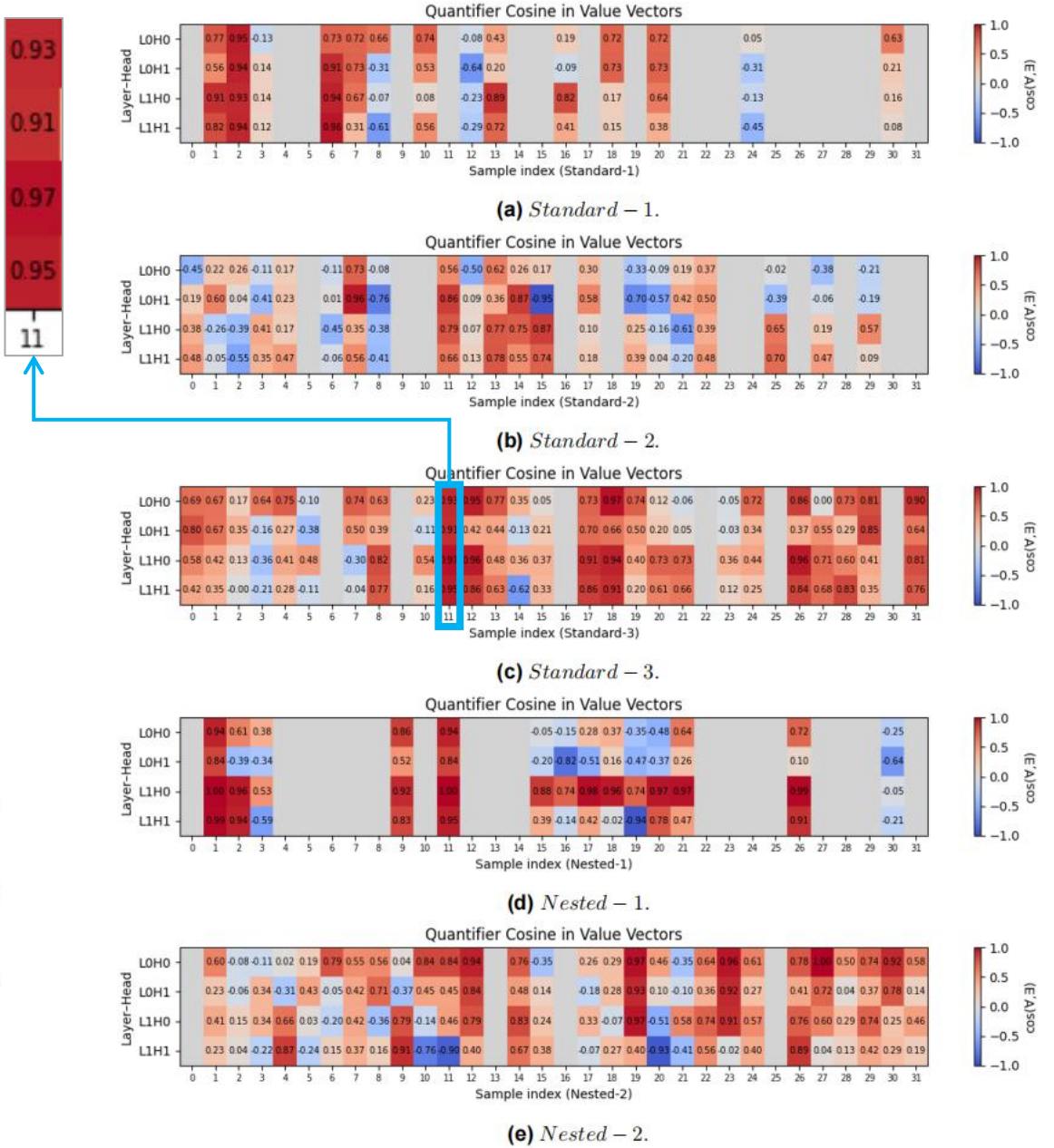


Figure 15. Value vectors of Sample 11 in $Standard-3$.



(a) $Standard-1$.

(b) $Standard-2$.

(c) $Standard-3$.

(d) $Nested-1$.

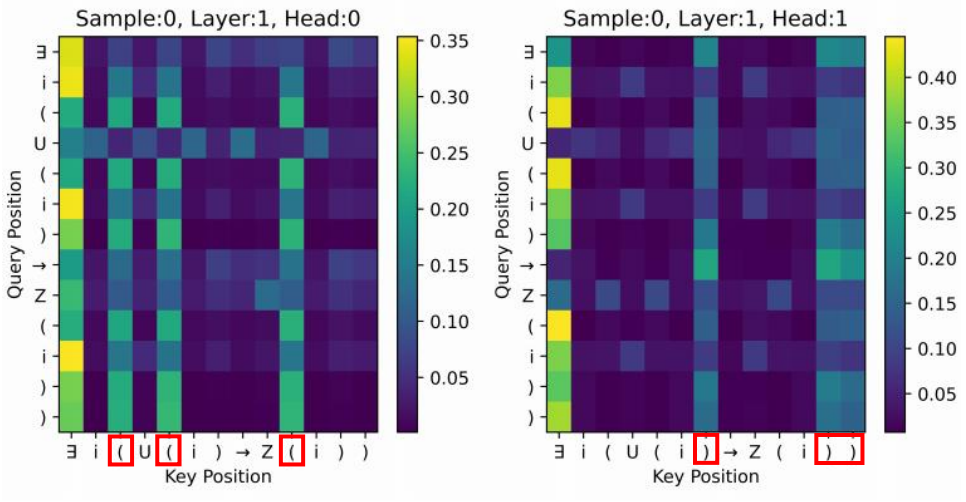
(e) $Nested-2$.

Figure 26. Cosine values of the mean angular distance on five validation datasets of a 2L2H Transformer with $d_{model} = 8$.

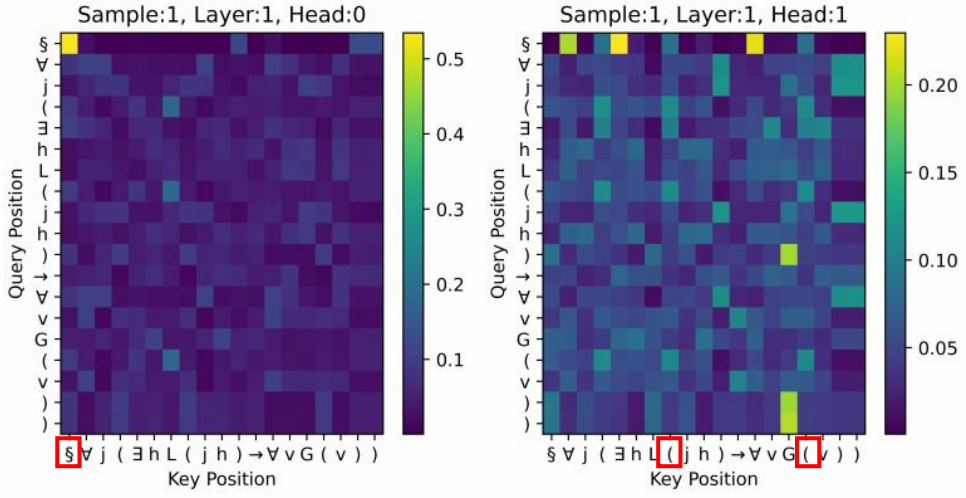
Interpretations

7-2-4 Key Information Aggregation

- **Mean pooling**
  - Whole seq → Task-relevant tokens
  - Two-stage Selection:
    - Global routing → Pairwise focuses
    - Distributed storage
  - Reconfirm each token's contribution to the output
- **BOS-token pooling**
  - BOS → Global readout
  - Less economical
- **Change behaviors of heads**



(a) Mean pooling.



(b) BOS-token pooling.

Figure 18. Attention heatmaps of two pooling strategies in Layer1 of a 2L2H Transformer on $Nested - 1$ with $QD = 1$.

7-2-4 Key Information Aggregation

- **Ebrahimi et al. (2020)[2], Weiss et al. (2021)[4]**
  - Dyck-k: BOS tokens → A stack base
  - Facilitate conditioned counting and Reduce the required attention heads

- **However**
  - A small, non-pretrained Transformer must first learn the special-token anchor and route information into it
    - BOS-pooled models require additional heads to reach 100% scores
    - RASP: need more heads to do aggregation into BOS

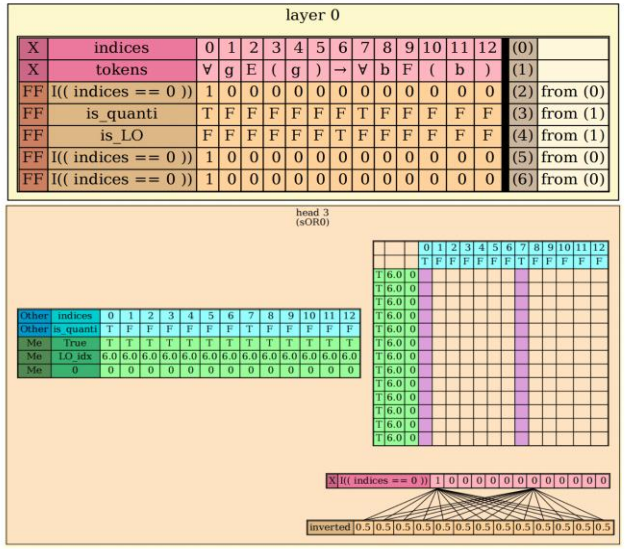
[2] Javid Ebrahimi, Dhruv Gelda, and Wei Zhang (2020). How Can Self-Attention Networks Recognize Dyck-n Languages? *Findings of the Association for Computational Linguistics: EMNLP 2020*. Association for Computational Linguistics, pp. 4301–4306.

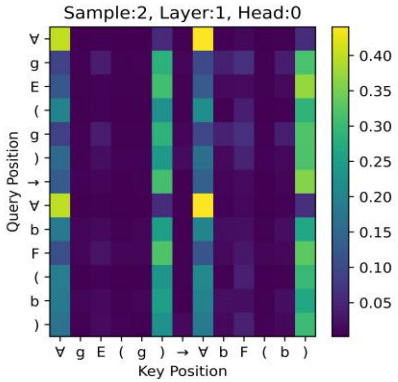
[4] Gail Weiss, Yoav Goldberg, and Eran Yahav (2021). 'Thinking Like Transformers'. In *Proceedings of the 38th International Conference on Machine Learning*. Vol. 139. PMLR, pp. 11080–11090.

Interpretations

7-2-4 Key Information Aggregation

- **Mean pooling**



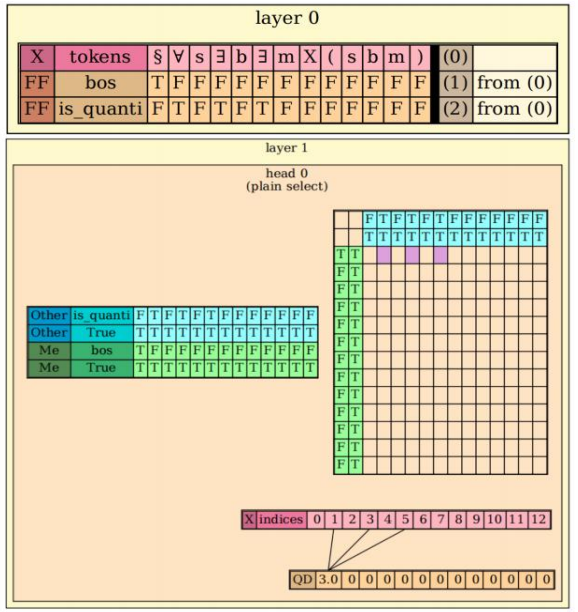


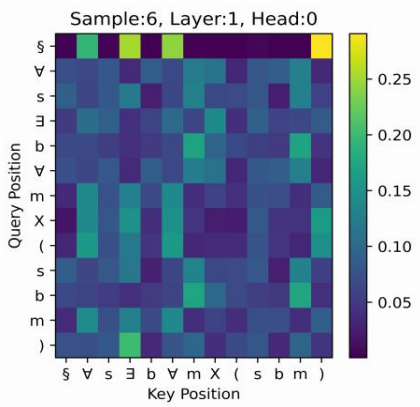
(a) In RASP visualisation.

(b) In a 2L2H Transformer.

Figure 16. A matching head of $Standard - 2$ when processing "$\forall gE(g) \rightarrow \forall bF(b)$".

- **BOS-token pooling**





(a) In RASP visualisation.

(b) In a 2L2H Transformer.

Figure 17. A matching head of $Standard - 1$ with BOS when processing "§$\forall s \exists b \exists m X(sbm)$".

Interpretations

7-2-4 Key Information Aggregation

- Pad → Auxiliary memory

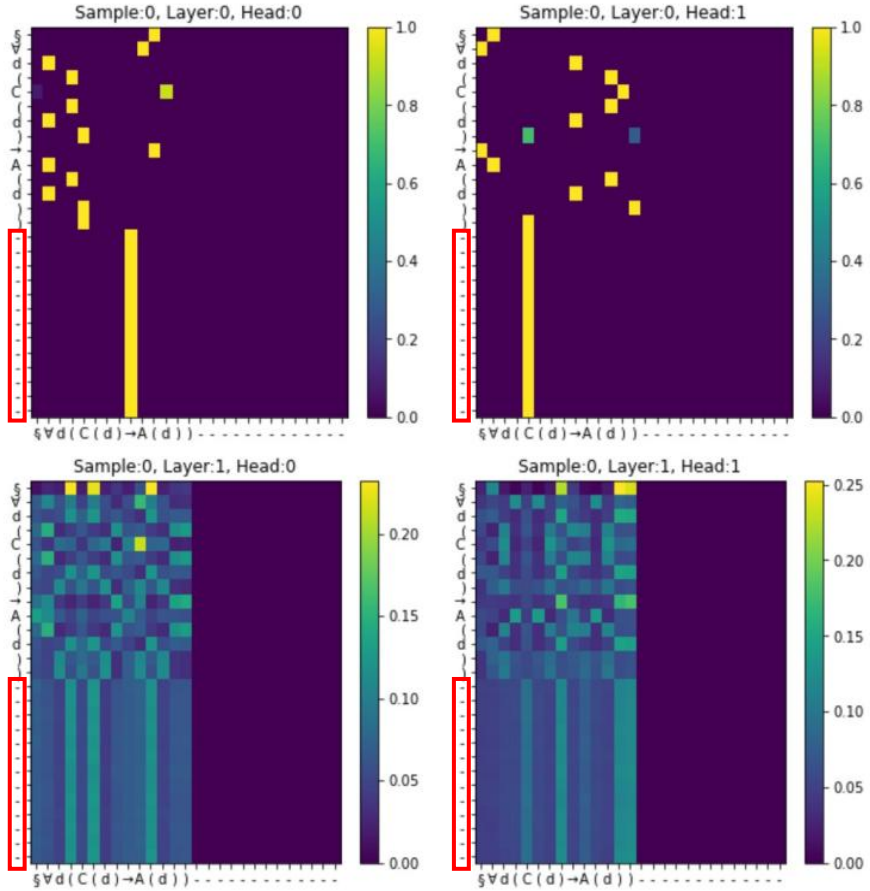


Figure 19. A heatmap example of a 2L2H Transformer with PAD on $Nested-1$ with $QD=1$, where "-" represents "<pad>".

Interpretations

7-2-5 Other Patterns

- Shortcut learning
  - Summarise the simplest rule from statistical co-occurrence

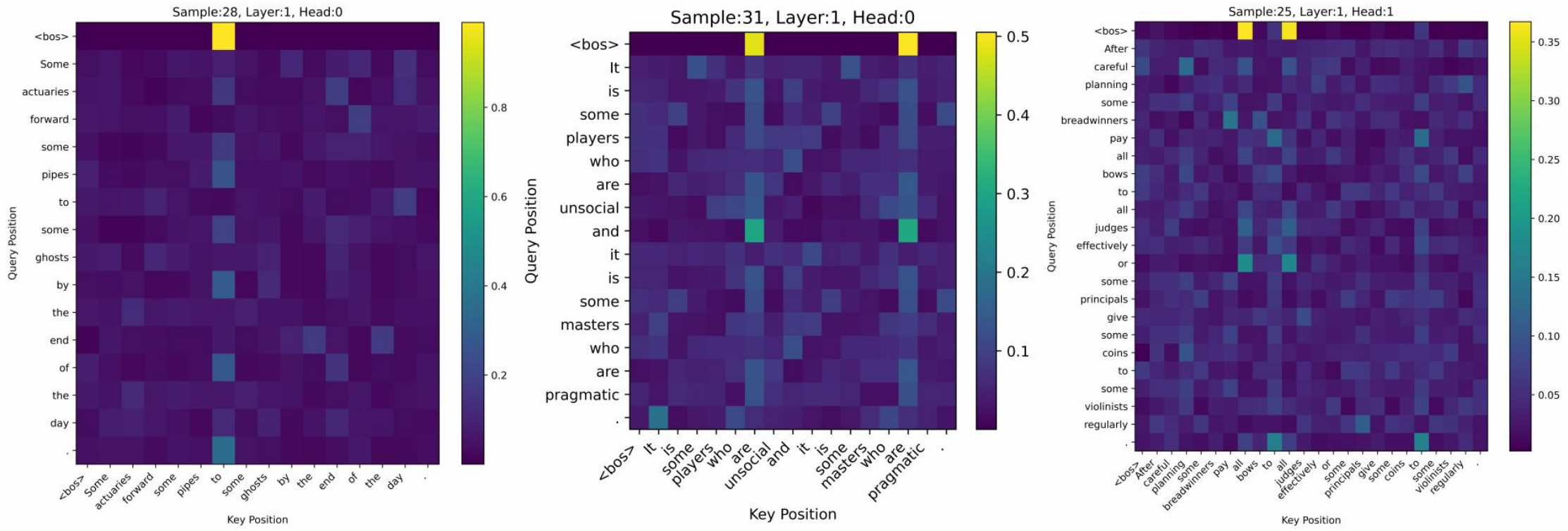
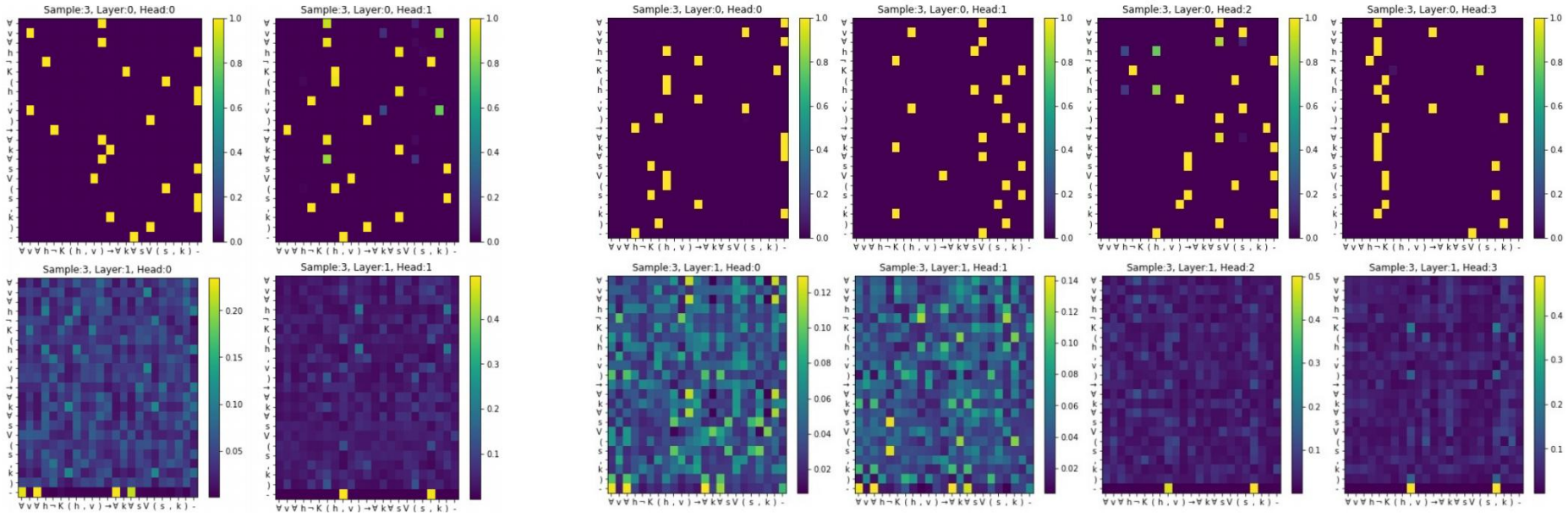


Figure 20. Three examples of a 2L2H Transformer with BOS on two FOL2NS datasets.

7-2-5 Other Patterns

- **Voita et al. (2019)[9], Michel et al. (2019)[10]**
  - Only a few specialised heads carry most of the workload
  - Others can be removed with a minor performance drop



(a) A 2L2H Transformer. **(b)** A 2L4H Transformer.

Figure 21. Same attention patterns in a 2L2H Transformer and a 2L4H Transformer.

[9] Paul Michel, Omer Levy, and Graham Neubig (2019). Are Sixteen Heads Really Better than One? In *Advances in Neural Information Processing Systems*. Vol. 32. Curran Associates, Inc.

[10] Elena Voita, David Talbot, Fedor Moiseev, Rico Sennrich, and Ivan Titov (2019). Analyzing Multi-Head Self-Attention: Specialized Heads Do the Heavy Lifting, the Rest Can Be Pruned. In *Proceedings of the 57th Annual Meeting of the Association for Computational Linguistics*. Association for Computational Linguistics, pp. 5797–5808.

Discussion

8 Theoretical vs. Empirical Results

- **Weak alignments in complex structures**
  - Different behaviors
  - Dismatch of upper bounds in RASP

- **RASP algorithms**
  1. Restricted attention
  2. Explicitly include a second pass in self-attention to recombine QD evidence across subformulas
  3. Based on positions
  4. Handle arbitrary quantifier nesting
  5. Fail to match on some simple tasks

     (e.g., sorting, Dyck-2)

- **Transformers**
  1. Soft attention
  2. Layer0: diffuse pairing; Layer1: appear some similar patterns; Defer sub_QD aggregation to subsequent components
  3. Extract all task-sepcific tokens in parallel
  4. Cannot process unbounded sequences

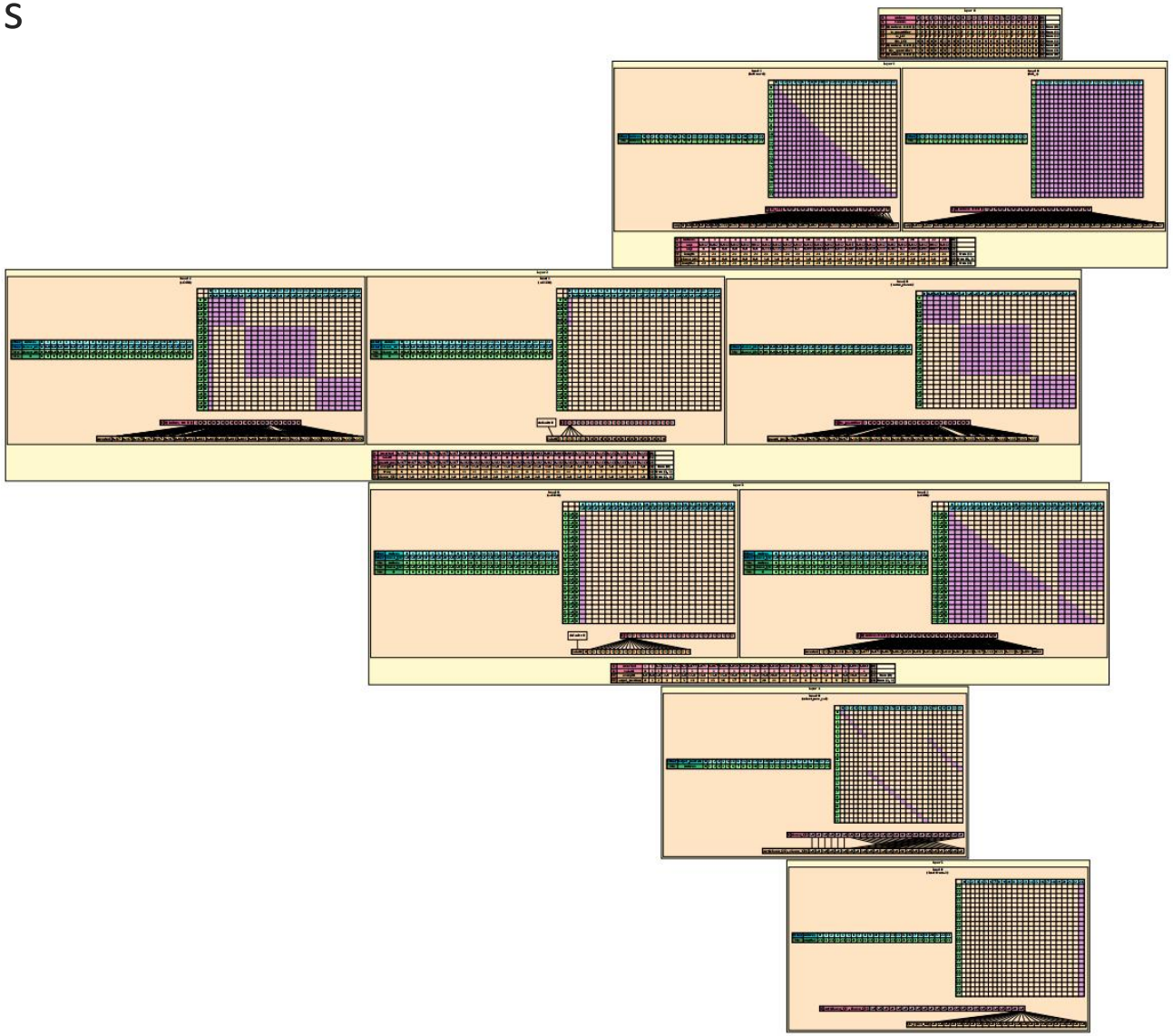
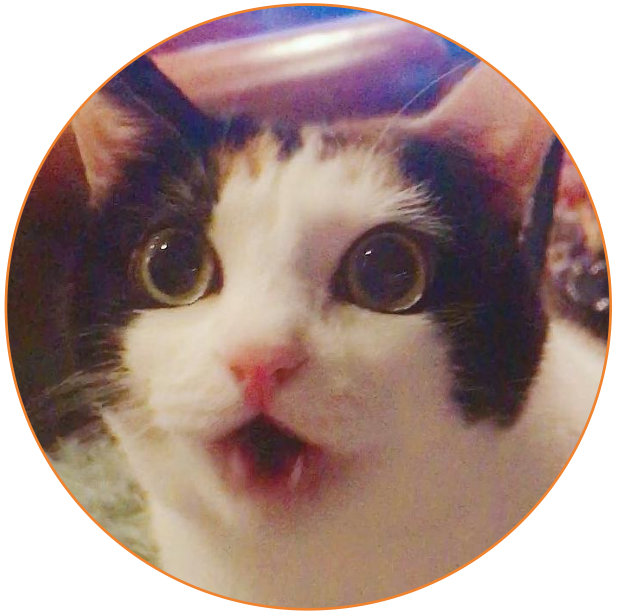


Figure 24. An example of the full RASP visualisation without BOS on $Standard - 3$ with $QD = 2$.

Q & A



Acknowledgements

Department of Linguistics and English Language

Colin Bannard (Senior Lecturer)

- ☑ **LELA60331** Computational Linguistics 1
- ☑ **LELA60341** Research Methods in CL 1
- ☑ **LELA60342** Research Methods in CL 2
- ☑ **LELA70502** Directed Reading 2
- ☑ Suggestions on logic-to-text generation

Andrea Nini (Senior Lecturer)

- ☑ **LELA60141** Foundational statistics with R
- ☑ Suggestions on academic writing

Richard Zimmermann (Lecturer)

- ☑ **LELA60112** Corpus Linguistics
- ☑ Suggestions on English syntax

Department of Computer Science

Chenghua Lin (Professor)

- ☑ **COMP61332** Text Mining
- ☑ **NLP Journal Club**
- ☑ Suggestions on logic-to-text generation

Jingyuan Sun (Lecturer)

Mingfei Sun (Lecturer)

- ☑ **COMP61011** Foundations of Machine Learning
- ☑ **RL Reading Group**

Edoardo Manino (Lecturer)

- ☑ **LLM Reading Group**

Ian Pratt-hartmann (Senior Lecturer)

- ☑ Suggestions on Formal Method & Logic

Konstantin Korovin (Reader)

External Inspiration

Gail Weiss (The author of RASP[4])

- ☑ Suggestions on RASP

[4] Weiss Gail, Goldberg Yoav and Yahav Eran. (2021). Thinking like Transformers. In *International Conference on Machine Learning (PMLR) 2021*, pages 11080-11090.

My Supervisor

Dmitry Nikolaev

Lecturer in Computational Linguistics

- [x] **LELA71000** Dissertation
- [x] **LELA60332** Computational Linguistics 2
- [x] **LELA60341** Research Methods in CL 1
- [x] **LELA60342** Research Methods in CL 2
- [x] **LELA71121** Directed Reading 1

- [x] Office Hour Regular Attendance
- [x] Email Flooding for Questions

Special Thanks to

My Cats (=Φ ω Φ=)

Main Mental Supports from China

Mimi
咪咪

Xuanyuan
轩辕

Anni
安妮

My Country

Happy the 76th Birthday to China!

1st October is the **China's National Day**.

MANCHESTER 1824
The University of Manchester

Thank you!